



Universidad de Jaén

Escuela Politécnica Superior de Jaén

Técnicas RAG para una asistente de docencia

Autor: José Juan Collado Araque

Grado: Ingeniería en Informática

**Directores: Manuel Carlos Díaz Galiano y
Departamento del director: Informática**

Fecha: 31/7/2026

Licencia CC



CREA



UNIVERSIDAD DE JAÉN

D./D^a Manuel Carlos Díaz Galiano y D./D^a , tutor(es) del Trabajo Fin de Grado titulado: **Técnicas RAG para una asistente de docencia**, que presenta José Juan Collado Araque, autoriza(n) su presentación para defensa y evaluación en la Escuela Politécnica Superior de Jaén.

Jaén, 31/7/2026

Agradecimientos

Este trabajo y mis estudios no hubieran sido posible sin el apoyo incondicional de mi familia y amigos. A ellos les agradezco el haber estado presentes en los momentos buenos, pero sobre todo en los malos.

Tabla de contenidos

1. INTRODUCCIÓN	1
1.1. Contexto y motivación	1
1.2. Importancia	1
1.3. Aportación	2
1.4. Objetivos	2
2. PLANIFICACIÓN	5
2.1. Alcance y dedicación	5
2.2. Metodología de desarrollo	6
2.3. Requisitos del sistema	8
2.4. Estructura de descomposición del trabajo	8
2.5. Planificación de los <i>sprints</i>	10
2.6. Calendario	12
2.7. Presupuesto	13
3. ANTECEDENTES	15
3.1. Evolución de los LLMs y aparición de RAG	15
3.1.1. Eliza (1966)	15
3.1.2. Modelos de lenguaje estadísticos (SLM)	17

3.1.3. Redes neuronales y Word Embeddings	17
3.1.4. Redes neuronales recurrentes (RNNs) (1986)	19
3.1.5. Memoria a largo-corto plazo (LSTM) (1997)	19
3.1.6. Transformers	19
3.1.7. Large language models (LLM)	20
3.2. La aparición de RAG	21
3.2.1. Componentes clave del RAG	22
3.2.2. Beneficios de RAG	23
3.2.3. Aplicaciones en el ámbito educativo	23
4. DISEÑO DE UN SISTEMA RAG	25
4.1. Cadena para la gestión de la documentación no estructurada (<i>Unstructured data pipeline</i>)	25
4.1.1. Selección e ingestión del corpus	26
4.1.2. Procesamiento de la documentación	26
4.1.3. <i>Chunking</i>	27
4.1.4. Embedding	29
4.2. Cadena RAG para la inferencia (<i>RAG chain for inference</i>)	31
4.2.1. Preprocesado de la consulta (<i>Query understanding</i>)	32
4.2.2. Recuperación (<i>Retrieval</i>)	33
4.2.3. Aumento del <i>prompt</i> (<i>Prompt augmentation</i>)	34
4.2.4. Generación (<i>LLM</i>)	34
4.2.5. Post-procesado	35
4.3. Evaluación y monitorización (<i>Evaluation and monitoring</i>)	36
4.3.1. Los dos tipos de calidad	36

4.3.2. Evaluación	36
4.3.3. Evaluar el rendimiento: métricas	37
5. MATERIALES Y MÉTODOS	39
5.1. Entorno y herramientas	40
5.1.1. Lenguaje de programación y entorno de ejecución	40
5.1.2. Framework	40
5.1.3. Proveedor de los modelos	41
5.2. Cadena para la gestión de la documentación	42
5.2.1. Selección del corpus	42
5.2.2. Conversión de PDF a Markdown	43
5.2.3. Limpieza de la documentación	45
5.2.4. <i>Chunking</i>	45
5.2.5. Embeddings	46
5.2.6. Base de datos vectorial	47
5.3. Cadena RAG para la inferencia	49
5.3.1. Preprocesado de la consulta	49
5.3.2. Recuperación	50
5.3.3. Aumento del <i>prompt</i>	52
5.3.4. Generación	53
5.3.5. Post-procesado	55
5.4. Evaluación y monitorización	56
5.4.1. Conjunto de evaluación	56
5.4.2. Herramienta de evaluación	57

5.4.3. Modelo juez	58
5.4.4. Comparación entre configuraciones	59
5.5. Resumen de las tecnologías elegidas	60
6. RESULTADOS	61
6.1. Cómo leer los resultados	61
6.1.1. Bancos de preguntas	61
6.1.2. Métricas	62
6.1.3. Configuración de referencia	62
6.1.4. El ruido de la medida	64
6.2. Rendimiento del sistema final	67
6.2.1. Dónde falla el sistema	68
6.3. Comparación de alternativas	70
6.3.1. Estilo de respuesta	70
6.3.2. Recuperación	71
6.3.3. Generador	74
7. CONCLUSIONES	77
7.1. Cumplimiento de los objetivos	77
7.2. Conclusiones principales	78
7.3. Limitaciones	79
7.4. Líneas de trabajo futuro	79
Bibliografía	XI

Lista de figuras

2.1. Diagrama de Gantt del proyecto, del 1 de julio al 30 de septiembre de 2026, con la dedicación de cada tarea y los cinco hitos.	12
3.1. Interfaz de Eliza (1966). Figura obtenida de http://deixilabs.com/eliza.html y conversación realizada por mí	16
3.2. Relación semántica entre palabras. Figura obtenida de [1]	18
3.3. Distancia entre palabras. Figura obtenida de [2]	18
3.4. Incremento de popularidad LLM. Figura obtenida de [3]	20
3.5. Aumento de capacidades LLM. Figura obtenida de [3]	21
3.6. Chunking. Figura obtenida de [4]	22
4.1. Unstructured data pipeline. Figura obtenida de [5]	25
4.2. Chunking. Figura obtenida de [5]	27
4.3. <i>Chunks</i> de tamaño fijo. Figura obtenida de https://chunkviz.up.railway.app/	29
4.4. <i>Comparación consulta-chunks</i> . Figura obtenida de [6]	30
5.1. Pipeline básico RAG. Figura obtenida de [7]	39
5.2. Indexación. Figura obtenida de [8]	43
5.3. Recuperación y generación de respuesta. Figura obtenida de [9]	49

- 6.1. Cambio de la corrección factual de cada pregunta entre dos evaluaciones en las que el modelo recibió exactamente los mismos *chunks*. En gris, las preguntas que repitieron nota, en azul, las que cambiaron. La parte sombreada la llamaremos banda de ruido, la cual alberga las notas que consideramos que han variado por obra del azar. 65
- 6.2. Corrección factual de cada una de las 94 preguntas del sistema final, según si los *chunks* recuperados contenían todo lo necesario para responder (contexto completo) o solo una parte (contexto parcial). Cada barra cuenta las preguntas con una nota en ese tramo. Los triángulos marcan la media de cada grupo. 69
- 6.3. Corrección factual frente al tiempo de respuesta, sin reordenado y con los dos modelos de reordenado evaluados. Cada punto es una evaluación completa de 94 preguntas, su altura es la corrección factual media, y su posición horizontal, la mediana del tiempo de respuesta. La franja gris es la banda de ruido alrededor de la opción elegida. 71
- 6.4. Efecto del número de *chunks* que llegan al generador sobre dos métricas de la recuperación (cobertura y precisión del contexto) y sobre la calidad de la respuesta (corrección factual). Media sobre las 94 preguntas. 73
- 6.5. Corrección factual (izquierda) y fidelidad (derecha) de los tres generadores, separando las preguntas según si los *chunks* recuperados contienen todo lo necesario para responder (contexto completo) o solo una parte (contexto parcial). Los tres generadores recibieron exactamente los mismos *chunks* en cada pregunta. 75

Lista de tablas

2.1. Adaptación de los elementos de Scrum a este proyecto.	7
2.2. Requisitos funcionales.	8
2.3. Requisitos no funcionales.	9
2.4. Estructura de descomposición del trabajo, con el periodo y la dedicación de cada tarea.	10
2.5. Planificación de los <i>sprints</i> semanales, con su objetivo, las horas previstas y los hitos.	11
2.6. Presupuesto estimado del proyecto.	13
5.1. Comparativa de herramientas de conversión de PDF a Markdown	44
5.2. Comparativa de bases de datos vectoriales	48
5.3. Métricas de evaluación utilizadas	57
5.4. Resumen de las tecnologías elegidas para cada parte del sistema	60
6.1. Tipos de pregunta del banco principal.	62
6.2. Métricas de la evaluación: qué mide cada una y cómo se obtiene. . . .	63
6.3. Configuración de referencia.	63
6.4. Resultados del sistema final sobre el banco de 94 preguntas. Valores entre 0 y 1, cuanto más alto mejor.	67
6.5. Resultados del sistema final según si el contexto recuperado estaba completo.	68

6.6. Diferencias entre los <i>prompts</i> de los dos estilos de respuesta.	70
6.7. Comparación de los dos estilos de respuesta sobre las 94 preguntas. La longitud es la mediana del número de palabras de cada respuesta; como comparación, se incluye la de las respuestas de referencia. . . .	70
6.8. Comparación de los tres generadores evaluados, con el mismo contex- to, el mismo <i>prompt</i> y el mismo juez.	74

Capítulo 1

INTRODUCCIÓN

1.1. Contexto y motivación

En los últimos años el acceso a la información ha sufrido un cambio radical gracias al auge de la inteligencia artificial. Hemos aumentado la cantidad de conocimiento que tenemos a nuestro alcance de forma exponencial. Por este constante aumento de documentación los estudiantes pueden verse sobrepasados.

Una forma de proporcionar a los estudiantes acceso a recursos educativos de calidad y apoyo académico personalizado es mediante el uso de sistemas *RAG (Retrieval-Augmented Generation)*. A diferencia de los modelos de lenguaje tradicionales que generan respuestas basadas únicamente en su conocimiento entrenado, un sistema *RAG* combina la recuperación de información relevante de una base de conocimiento con la generación de respuestas coherentes y contextualizadas como se explica en [\[10\]](#).

1.2. Importancia

Los sistemas *RAG* pueden llegar a ser particularmente valiosos en el ámbito educativo, donde es crucial que las respuestas se basen en la documentación oficial de la asignatura. Uno de los puntos fuertes de este sistema es la trazabilidad que nos puede llegar a aportar, explicado con detalle en el artículo es decir, podremos conocer que partes exactas de nuestra base de conocimiento ha utilizado el sistema para darnos la respuesta. De esta forma podremos explorar la fuente en si y evaluar la calidad de la respuesta por nosotros mismos. Swacha y Gracel hablan de «trazabilidad pedagó-

gica» en su artículo [11], la cual resulta especialmente útil en documentación técnica donde la precisión de las respuestas sea crucial.

1.3. Aportación

Este trabajo pretende realizar las siguientes aportaciones:

Evaluación sistemática de arquitecturas RAG en contexto educativo: Se proporcionará un análisis detallado de cómo diferentes configuraciones de RAG impactan en la calidad de las respuestas generadas.

Desarrollo de un prototipo funcional: Se implementará un sistema RAG completamente funcional que puede servir como referencia para futuras implementaciones.

Metodología de evaluación: Se definirá un conjunto de métricas y procedimientos para evaluar la efectividad de asistentes RAG en el ámbito educativo.

Identificación de limitaciones y oportunidades: El trabajo identificará tanto los puntos fuertes como las limitaciones de esta aproximación, así como futuras líneas de investigación.

1.4. Objetivos

El objetivo principal de este trabajo es **desarrollar un sistema RAG** capaz de facilitar el estudio de una asignatura académica, proporcionando respuestas coherentes y precisas a partir de la documentación oficial de la misma. Para conseguirlo se identifican los siguientes objetivos específicos, cada uno de los cuales aborda un desafío técnico concreto del sistema.

Estudio Teórico

Investigar la arquitectura RAG, sus componentes (módulos de recuperación y de generación), las técnicas de vectorización de tokens (embeddings), y los modelos de lenguaje (LLMs) más adecuados para esta tarea.

Definición de Requisitos

Establecer los requisitos funcionales y no funcionales del sistema tutor RAG, incluyendo la selección de la asignatura de estudio y la documentación a utilizar (apuntes,

libros de texto, etc.).

Implementación del Prototipo

Desarrollar un demostrador de un tutor RAG, que permita a los usuarios interactuar con él y realizar preguntas sobre la asignatura. Se diseñará una interfaz de usuario básica para facilitar las pruebas.

Diseño de Pruebas

Crear un conjunto de pruebas sistemáticas para evaluar la precisión, coherencia y utilidad de las respuestas del sistema RAG. Estas pruebas incluirán preguntas de diferentes niveles de complejidad (desde preguntas de definición hasta preguntas de razonamiento).

Análisis de Resultados

Analizar los resultados obtenidos en las pruebas, identificando las fortalezas y debilidades del sistema. Se medirá la precisión de las respuestas, la capacidad para manejar ambigüedad y la latencia del sistema.

Conclusiones y Futuras Líneas

Extraer conclusiones sobre la viabilidad y efectividad del uso de RAG como herramienta de tutorización, y proponer futuras líneas de investigación para mejorar el prototipo.

Capítulo 2

PLANIFICACIÓN

En este capítulo se planifica el proyecto antes de abordar su desarrollo. Siguiendo la guía PMBOK [12], la planificación de un proyecto responde a tres preguntas: qué hay que hacer (el alcance), cuándo (el tiempo) y cuánto cuesta (el coste). Las secciones del capítulo responden a ellas en ese orden: primero se fija la dedicación y el alcance del trabajo, después la metodología con la que se va a organizar, los requisitos que debe cumplir el sistema y el desglose de las tareas, a continuación el calendario, y por último el presupuesto.

2.1. Alcance y dedicación

El Trabajo Fin de Grado del Grado en Ingeniería Informática de la Universidad de Jaén tiene una carga de **12 créditos ECTS**. Según el Real Decreto 1125/2003 [13], cada crédito europeo corresponde a entre 25 y 30 horas de trabajo del estudiante. La guía docente de la asignatura concreta esta regla: «12 créditos ECTS del TFG suponen 300 horas de trabajo del estudiante», que implican además al menos 4 horas de orientación individualizada del tutor [14].

Por tanto, el proyecto se planifica con una dedicación de **300 horas**, repartidas en **tres meses**, del 1 de julio al 30 de septiembre de 2026. Son 13 semanas, lo que supone una media de unas 23 horas de trabajo por semana.

El alcance del proyecto queda delimitado de la siguiente forma:

- **Dentro del alcance:** un sistema RAG que responde a preguntas sobre la asignatura de Sistemas Operativos a partir de su documentación oficial, citando sus

fuentes; una interfaz de usuario para utilizarlo, y una evaluación sistemática de su calidad, que incluye la comparación de distintas configuraciones.

- **Fuera del alcance:** el despliegue del sistema en producción para todos los estudiantes, su adaptación a otras asignaturas y el entrenamiento o ajuste de modelos de lenguaje propios.

2.2. Metodología de desarrollo

Para organizar el trabajo se ha elegido **Scrum** [15], un marco de trabajo ágil en el que el producto se construye en ciclos cortos de duración fija, llamados *sprints*. Al final de cada *sprint* se obtiene una versión del producto que funciona (un *incremento*), se revisa y se decide qué hacer a continuación.

Este enfoque encaja con las características del proyecto. Los modelos de lenguaje y las herramientas que los rodean evolucionan muy deprisa, y a priori no se sabe qué combinación de técnicas dará mejores resultados con el temario de la asignatura. En lugar de diseñar todo el sistema de antemano, Scrum permite construir pronto una versión sencilla que funcione de extremo a extremo y mejorarla después, decidiendo en cada ciclo qué parte conviene cambiar a la vista de lo medido.

Scrum está pensado para equipos de varias personas, por lo que se ha adaptado a un proyecto individual. La adaptación más importante tiene que ver con el ritmo de trabajo: **cada semana, el estudiante y el tutor se reúnen para revisar el avance realizado y acordar qué debe completarse la semana siguiente**. Esa reunión semanal marca la duración de los *sprints*, que es de una semana, y reúne en un solo encuentro la revisión del *sprint* que termina y la planificación del siguiente. La tabla 2.1 resume cómo se ha adaptado cada elemento de Scrum.

Para decidir cuándo una tarea está terminada se aplica una **definición de hecho** común a todo el proyecto. Una tarea se considera hecha cuando:

- el sistema funciona de extremo a extremo con el cambio incorporado;
- supera las pruebas automáticas;
- se ha medido su efecto con la evaluación, cuando el cambio afecta a la calidad de las respuestas, y
- está documentada en la memoria.

Tabla. 2.1: Adaptación de los elementos de Scrum a este proyecto.

Elemento de Scrum	Adaptación en este proyecto
<i>Roles</i>	
Propietario del producto	El tutor, que representa las necesidades de estudiantes y profesorado, y prioriza el trabajo.
Equipo de desarrollo y Scrum Master	El estudiante, que desarrolla el sistema y se encarga de que el proceso se cumpla.
<i>Eventos</i>	
<i>Sprint</i>	Una semana. En total, 13 <i>sprints</i> .
Revisión y planificación del <i>sprint</i>	La reunión semanal con el tutor: se revisa lo realizado durante la semana y se acuerda qué completar en la siguiente.
Retrospectiva	Unos minutos al final de la reunión semanal para comentar qué ha funcionado y qué conviene cambiar en la forma de trabajar.
Reunión diaria	Al ser un proyecto individual, se sustituye por un registro de trabajo en el que se anotan las tareas realizadas y los problemas encontrados.
<i>Artefactos</i>	
Pila del producto	Los requisitos de la sección 2.3, ordenados por prioridad.
Pila del <i>sprint</i>	Las tareas acordadas en la reunión semanal.
Incremento	Una versión ejecutable del sistema al final de cada semana.

2.3. Requisitos del sistema

Los requisitos describen qué debe hacer el sistema (requisitos funcionales) y con qué características debe hacerlo (requisitos no funcionales). Se han redactado siguiendo las recomendaciones de la norma ISO/IEC/IEEE 29148 [16]: cada requisito tiene un identificador único, expresa una única necesidad y, en el caso de los no funcionales, va acompañado de un criterio que permite comprobar si se cumple. En Scrum, estos requisitos forman la pila del producto inicial: la prioridad indica el orden en que se abordan y la última columna, el *sprint* en el que está previsto completarlos.

Tabla. 2.2: Requisitos funcionales.

ID	Requisito	Prioridad	<i>Sprint</i>
RF-01	Convertir los documentos PDF del corpus a Markdown conservando su estructura, procesando solo los documentos nuevos o modificados.	Alta	S5–S8
RF-02	Trocear el corpus en fragmentos e indexarlos en una base de datos vectorial.	Alta	S6–S8
RF-03	Responder en lenguaje natural a preguntas sobre la asignatura utilizando únicamente la información del corpus.	Alta	S6–S9
RF-04	Citar el fragmento del que procede cada afirmación de la respuesta y mostrar las fuentes al usuario.	Alta	S9
RF-05	Responder con una frase de rechazo fija cuando la información solicitada no se encuentre en el corpus.	Alta	S9
RF-06	Mantener una conversación, entendiendo las preguntas de seguimiento que hacen referencia a mensajes anteriores.	Media	S9
RF-07	Consultar desde la interfaz los documentos originales del corpus.	Media	S11–S12
RF-08	Inspeccionar paso a paso las etapas de la recuperación de una pregunta, como herramienta de diagnóstico.	Baja	S7–S9
RF-09	Evaluar automáticamente el sistema con un banco de preguntas y un conjunto de métricas.	Alta	S8–S10
RF-10	Comparar dos configuraciones del sistema para medir el efecto de un cambio.	Media	S11–S13

2.4. Estructura de descomposición del trabajo

Para estimar el esfuerzo y el calendario, el proyecto se descompone en paquetes de trabajo y estos, a su vez, en tareas. Esta estructura de descomposición del trabajo (EDT) es una de las herramientas básicas de la guía PMBOK [12]: permite asignar a cada tarea una dedicación y unas fechas, y comprobar que la suma de todas ellas se

Tabla. 2.3: Requisitos no funcionales.

ID	Requisito	Criterio de aceptación
RNF-01	Coste	El uso del sistema no tiene coste por consulta: todos los servicios que utiliza son gratuitos.
RNF-02	Fidelidad	Las respuestas se basan únicamente en el corpus. Ante preguntas cuya respuesta no está en el temario, el sistema responde con la frase de rechazo.
RNF-03	Rendimiento	La mediana del tiempo de respuesta es inferior a 10 segundos en un ordenador sin tarjeta gráfica.
RNF-04	Idioma	El sistema responde en español, el idioma de la asignatura.
RNF-05	Reproducibilidad	Cada evaluación registra la configuración con la que se realizó, y los modelos se fijan por un identificador concreto que no cambie entre evaluaciones.
RNF-06	Portabilidad	El sistema se ejecuta en un ordenador portátil sin tarjeta gráfica.
RNF-07	Mantenibilidad	La interfaz y la evaluación comparten un mismo núcleo, cubierto por pruebas automáticas.
RNF-08	Seguridad	Las claves de acceso a las APIs no se incluyen en el código fuente, y la interfaz web solo es accesible desde el propio equipo.
RNF-09	Usabilidad	La interfaz web es sencilla y muestra la respuesta a medida que se genera.

ajusta a las horas disponibles. La tabla 2.4 recoge los seis paquetes de trabajo y sus once tareas.

Los paquetes siguen el orden lógico del proyecto. El **estudio** recoge la investigación del estado del arte, que se presenta en el capítulo de Antecedentes, y la comparación de tecnologías para cada parte del sistema. El **análisis** fija los requisitos de la sección anterior y selecciona la documentación de la asignatura que formará el corpus. El **desarrollo** empieza por un prototipo sencillo que funcione de extremo a extremo y continúa con la versión definitiva de cada parte: la cadena que prepara la documentación, la cadena RAG que responde a las preguntas y la interfaz. La **evaluación** construye el banco de preguntas, mide la calidad del sistema y compara distintas configuraciones. Por último, la **gestión** y la **documentación** son transversales y se extienden a lo largo de todo el proyecto.

Tabla. 2.4: Estructura de descomposición del trabajo, con el periodo y la dedicación de cada tarea.

Paquete y tarea	Periodo	Horas
1. Gestión del proyecto		15
1.1 Reuniones semanales y seguimiento	1 jul – 30 sep	15
2. Estudio		45
2.1 Estado del arte	1 jul – 21 jul	30
2.2 Estudio de tecnologías	15 jul – 4 ago	15
3. Análisis		12
3.1 Requisitos y selección del corpus	8 jul – 21 jul	12
4. Desarrollo		140
4.1 Prototipo de extremo a extremo	22 jul – 11 ago	30
4.2 Cadena de datos	29 jul – 25 ago	45
4.3 Cadena RAG	12 ago – 8 sep	40
4.4 Interfaz de usuario	2 sep – 22 sep	25
5. Evaluación		53
5.1 Banco de preguntas y evaluación automática	19 ago – 8 sep	25
5.2 Experimentos y comparativas	9 sep – 30 sep	28
6. Documentación		35
6.1 Redacción de la memoria	1 jul – 30 sep	35
Total		300

2.5. Planificación de los *sprints*

La tabla 2.5 reparte el trabajo en los 13 *sprints* semanales. Para cada uno se indica el objetivo que se revisará en la reunión con el tutor al final de la semana y las horas previstas, que se obtienen repartiendo la dedicación de cada tarea de la EDT por igual entre los días que dura. Cinco de los *sprints* terminan con un **hito**: un punto de control en el que debe estar disponible un resultado concreto del proyecto.

Los cinco hitos del proyecto son:

- **H1, 21 de julio**: requisitos definidos y pila del producto aprobada por el tutor.
- **H2, 11 de agosto**: primer prototipo funcional, capaz de responder preguntas de extremo a extremo.
- **H3, 8 de septiembre**: primera evaluación completa del sistema con el banco de preguntas.
- **H4, 22 de septiembre**: sistema completo, con la interfaz de usuario.
- **H5, 30 de septiembre**: evaluación final y memoria terminada.

Tabla. 2.5: Planificación de los *sprints* semanales, con su objetivo, las horas previstas y los hitos.

<i>Sprint</i>	Fechas	Objetivo de la semana	Horas
S1	1 jul – 7 jul	Visión del producto e inicio del estudio del estado del arte.	14
S2	8 jul – 14 jul	Estado del arte e identificación de los requisitos.	20
S3	15 jul – 21 jul	Requisitos, selección del corpus y pila del producto. Hito H1.	25
S4	22 jul – 28 jul	Estudio de tecnologías y arranque del prototipo.	19
S5	29 jul – 4 ago	Prototipo y conversión de los documentos a Markdown.	30
S6	5 ago – 11 ago	Prototipo funcionando de extremo a extremo. Hito H2.	25
S7	12 ago – 18 ago	Troceado e indexado del corpus.	25
S8	19 ago – 25 ago	Cadena de datos definitiva y banco de preguntas.	33
S9	26 ago – 1 sep	Recuperación y generación de respuestas citadas.	22
S10	2 sep – 8 sep	Primera evaluación completa del sistema. Hito H3.	30
S11	9 sep – 15 sep	Interfaz web y primeras comparativas.	21
S12	16 sep – 22 sep	Sistema completo con interfaz. Hito H4.	21
S13	23 sep – 30 sep	Comparativas finales y cierre de la memoria. Hito H5.	15
Total			300

2.6. Calendario

La figura 2.1 muestra el calendario del proyecto en un diagrama de Gantt. Cada fila corresponde a una tarea de la EDT, con su dedicación entre paréntesis; las barras negras agrupan las tareas de cada paquete, y los rombos marcan los hitos. Las dos filas superiores indican los meses y los *sprints* semanales.

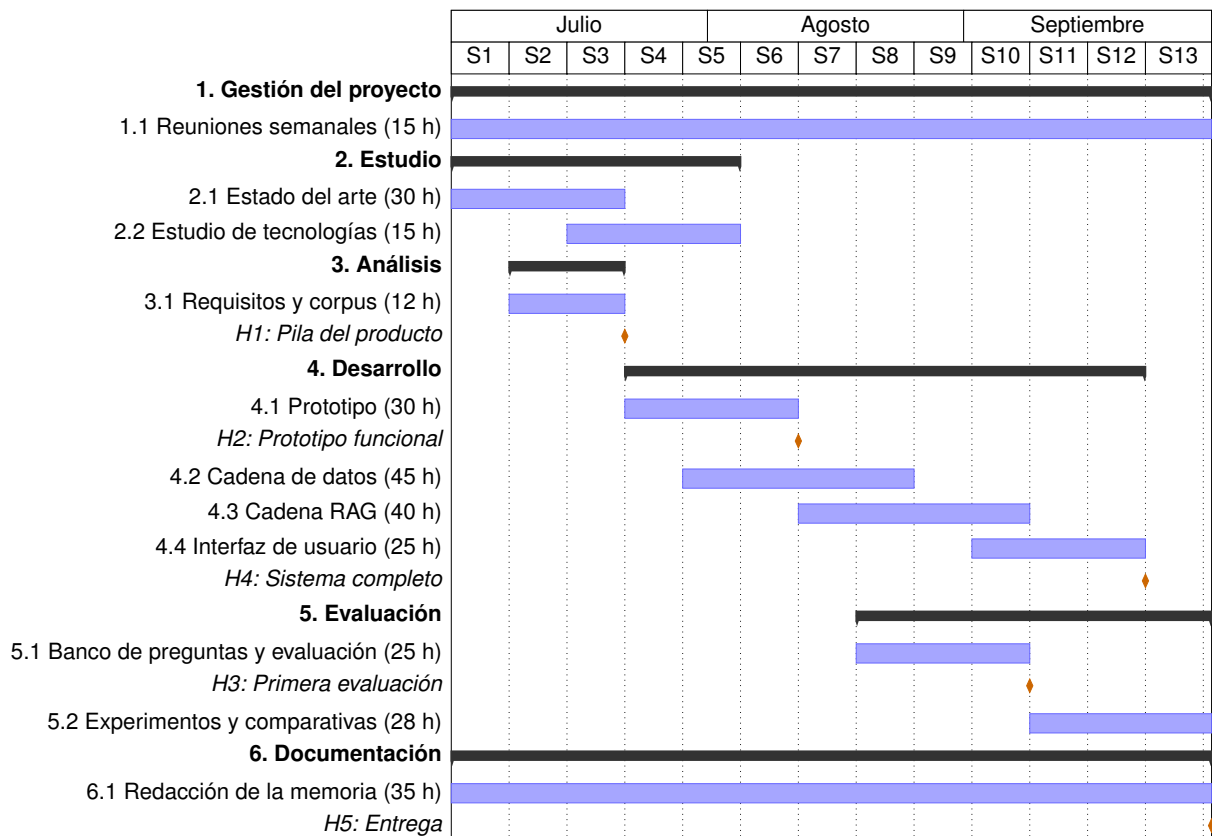


Figura 2.1: Diagrama de Gantt del proyecto, del 1 de julio al 30 de septiembre de 2026, con la dedicación de cada tarea y los cinco hitos.

Como se observa en el diagrama, muchas tareas se solapan. Es una consecuencia directa de la metodología: el prototipo empieza a construirse mientras todavía se estudian las tecnologías, y la cadena de datos se sigue mejorando cuando ya se trabaja en la cadena RAG, porque cada reunión semanal puede revelar un problema en una parte ya construida. La carga de trabajo es mayor en la parte central del proyecto, entre finales de julio y principios de septiembre, cuando coinciden el desarrollo de las dos cadenas y la construcción de la evaluación. La gestión y la redacción de la memoria se extienden a lo largo de las 13 semanas.

2.7. Presupuesto

La normativa de Trabajos Fin de Título de la Escuela Politécnica Superior de Jaén contempla el presupuesto como parte de la documentación de los trabajos de tipo proyecto de ingeniería [17]. Aunque el sistema se ha diseñado para no tener coste de uso, su desarrollo sí lo tiene. La tabla 2.6 lo estima como si el proyecto se realizara en un entorno profesional, con las siguientes suposiciones:

- Las 300 horas del estudiante se valoran según el coste para la empresa de un ingeniero informático junior, estimado en 20 euros por hora.
- La dedicación del tutor, valorada a 40 euros por hora, comprende las 13 reuniones semanales, de alrededor de una hora, y unas 7 horas de revisión de la memoria. Supera así las 4 horas mínimas de orientación que fija la guía docente.
- El ordenador portátil, de 1 000 euros y con una vida útil de cuatro años, se imputa por su amortización durante los tres meses del proyecto.
- El software utilizado es de código abierto y los modelos de lenguaje se consumen a través de un servicio académico gratuito. El único servicio de pago es un crédito prepago de 5 dólares en la API de OpenAI, previsto para comparar el sistema con un generador comercial, convertido a un tipo de cambio de 0,92 euros por dólar.

Tabla. 2.6: Presupuesto estimado del proyecto.

Concepto	Cantidad	Precio unitario	Importe
<i>Personal</i>			
Desarrollo (ingeniero junior)	300 h	20,00 €/h	6 000,00 €
Dirección y revisión (tutor)	20 h	40,00 €/h	800,00 €
<i>Equipos, software y servicios</i>			
Ordenador portátil (amortización)	3 meses	20,83 €/mes	62,50 €
Software de código abierto	—	Gratuito	0,00 €
Modelos de lenguaje (servicio académico)	—	Gratuito	0,00 €
Crédito de la API de OpenAI	5,00 \$	0,92 €/ \$	4,60 €
Electricidad e internet (proporcional)	3 meses	15,00 €/mes	45,00 €
Subtotal			6 912,10 €
Costes indirectos (10 %)			691,21 €
Total (IVA no incluido)			7 603,31 €

El coste del proyecto se debe casi por completo al personal, que supone el 98,4 % del subtotal. Gracias al uso de software de código abierto y de modelos gratuitos,

el gasto en equipos y servicios es mínimo, y una vez desarrollado el sistema puede utilizarse sin ningún coste por consulta, tal y como exige el requisito RNF-01.

Capítulo 3

ANTECEDENTES

El lenguaje es una capacidad humana fundamental para la expresión y la comunicación, la cual se desarrolla en la infancia y evoluciona a lo largo de toda la vida. Las máquinas, por el contrario, no poseen de forma natural la habilidad de comprender o comunicarse mediante el lenguaje humano, a menos que estén dotadas con inteligencia artificial. Lograr hacer que las máquinas lean, escriban y se comuniquen de manera similar a los seres humanos ha constituido un desafío de investigación histórico que persiste hasta día de hoy.

Desde un punto de vista técnico, los modelos del lenguaje (*LM*) constituye uno de los mayores acercamientos que se han tenido para conseguir la inteligencia lingüística en las máquinas. En términos generales, los *LM* tienen como objetivo modelar la probabilidad generativa de las secuencias de palabras, con el fin de predecir la siguiente palabra que tenga mayores probabilidades de encajar dentro de su contexto.

3.1. Evolución de los LLMs y aparición de RAG

3.1.1. Eliza (1966)

En 1966 el mundo presenció el nacimiento Eliza, el cual es considerado ser el primer chatbot creado. Según Berry y David M. en su artículo *The Limits of Computation: Joseph Weizenbaum and the ELIZA Chatbot* [18] Eliza fue un experimento revolucionario en su momento ya que asentó las bases de la interacción persona-ordenador. Este chatbot no entendía el contexto de lo que se estaba hablando como lo hacen los LLMs de hoy en día, sino que creaba la ilusión de que se estaba teniendo una conver-

sación. Esto se logró gracias a la reformulación de las frases dadas por el usuario en forma de pregunta.

Eliza promovió la investigación en el campo de los chatbots y el procesamiento del lenguaje natural en los años venideros. Una de las variantes de Eliza más destacables es *Doctor*. Pese a que Eliza fue un comienzo humilde y contenido nos ayuda a poner en perspectiva la escala a la cual hemos llegado en los últimos años.

Eliza

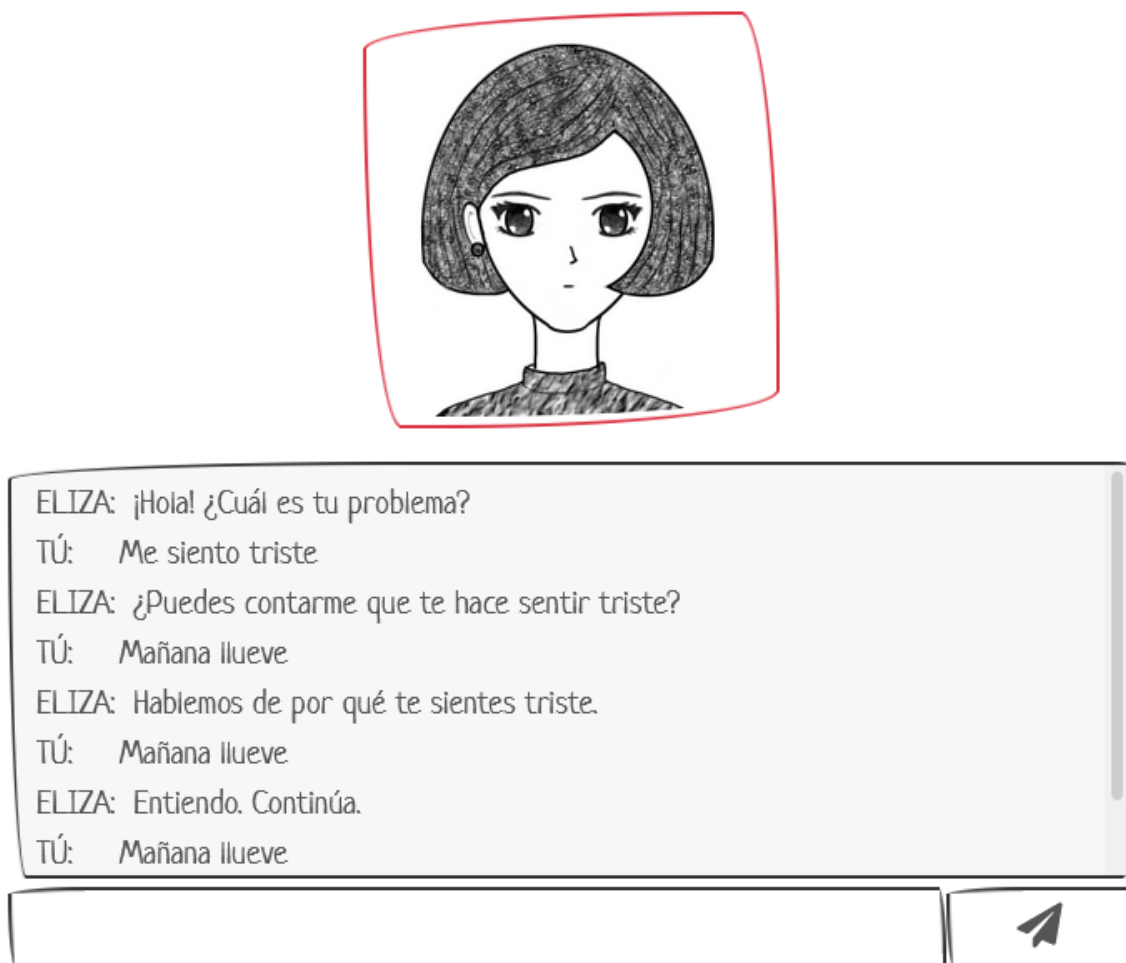


Figura 3.1: Interfaz de Eliza (1966). Figura obtenida de <http://deixilabs.com/eliza.html> y conversación realizada por mí

Como podemos ver en la figura 3.1, Eliza tiene problemas evidentes para dar respuestas con sentido y basándose en el contexto.

3.1.2. Modelos de lenguaje estadísticos (SLM)

Los primeros modelos de lenguaje fueron desarrollados hacia los años 90 y como bien se explica en la obra de Wayne Xin Zhao [3] la idea era construir modelos capaces de predecir la siguiente palabra en una secuencia de texto usando diferentes técnicas estadísticas.

El problema de estos primeros modelos es que eran entrenados con conjuntos de datos (que se conocen como corpus) relativamente pequeños, y además las capacidades de cómputo eran limitadas.

Esto hacía que los modelos estadísticos no lograran captar de forma adecuada las diferentes relaciones existentes entre los diferentes elementos del texto y por tanto su capacidad predictiva era muy limitada. No funcionaban bien en la práctica.

3.1.3. Redes neuronales y Word Embeddings

Según Nielsen en su obra [19] una red neuronal es un sistema matemático que aprende a reconocer patrones ajustando iterativamente un conjunto de parámetros: los *pesos* y los *sesgos*. Su unidad básica es la neurona sigmoide, que recibe varias entradas numéricas, calcula su suma ponderada y aplica una función sigmoide.

A diferencia del perceptrón original, cuya salida era binaria y discontinua, la función sigmoide es suave y derivable, lo que permite que pequeños cambios en los pesos produzcan pequeños cambios en la salida. Esta propiedad es la que hace posible el aprendizaje mediante el descenso del gradiente.

Las neuronas se organizan en capas de entrada, ocultas y de salida formando redes *feedforward* capaces de aproximar cualquier función continua con suficiente profundidad y anchura.

Para que estas redes puedan trabajar con texto, es necesario transformar las palabras en vectores numéricos. El enfoque clásico de *one-hot encoding* asignaba a cada palabra un vector con un único valor 1 en su posición del vocabulario, sin capturar ninguna relación semántica entre palabras. En 2013, Mikolov et al. propusieron Word2Vec [20], un modelo que sitúa cada palabra en un espacio vectorial continuo de cientos de dimensiones donde la proximidad geométrica refleja similitud semántica. El resultado más célebre de este enfoque es que operaciones algebraicas sobre los vectores capturan relaciones del mundo real como podemos ver en la figura 3.2.

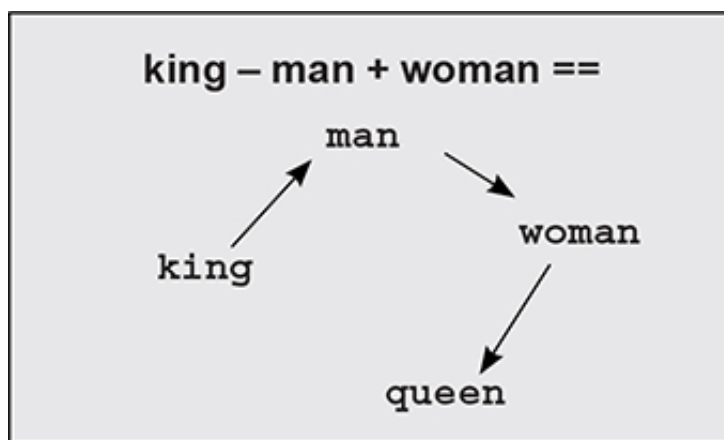


Figura 3.2: Relación semántica entre palabras. Figura obtenida de [1]

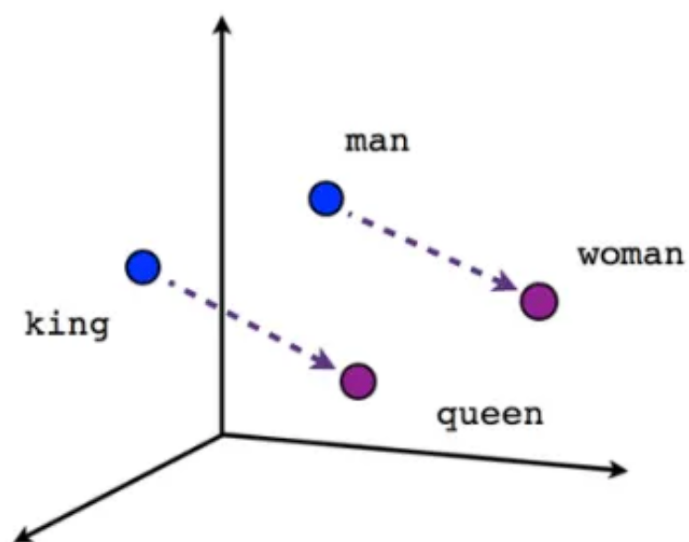


Figura 3.3: Distancia entre palabras. Figura obtenida de [2]

En la figura 3.3 podemos ver de una forma más visual la distancia que existe entre conceptos.

Word2Vec se entrenaba mediante dos arquitecturas complementarias: **CBOW**, que predice una palabra a partir de su contexto, y **Skip-gram**, que hace lo inverso. Estos embeddings estáticos sentaron las bases de la representación vectorial del lenguaje que utilizan los modelos actuales.

3.1.4. Redes neuronales recurrentes (RNNs) (1986)

Según Alex Sherstinsky en su artículo *Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) Network* [21] en los años 80 surgen las redes neuronales inspiradas por el cerebro humano y las interconexiones entre sus neuronas. Sobre estas se construyen las redes neuronales recurrentes (RNNs) en 1986 ganando mucha popularidad en poco tiempo. La particularidad de los RNNs es que podían recordar información previa registrada en su memoria gracias a que tenían un bucle de retroalimentación hacendolas adecuadas para el procesamiento de lenguaje natural.

3.1.5. Memoria a largo-corto plazo (LSTM) (1997)

Volvemos a recurrir a la obra de Alex Sherstinsky [21] para destacar que los LSTMs son una especialización de los RNNs que vienen a superar las limitaciones de la memoria a corto plazo, dotandolos con la habilidad de recordar información sobre secuencias mucho más largas.

Esto se consigue mediante la incorporación de un mecanismo interno basado en puertas (gates) que permite controlar el flujo de información a lo largo de la secuencia.

3.1.6. Transformers

En 2017, Vaswani et al. publicaron *Attention Is All You Need* [22], proponiendo una arquitectura que prescindía por completo de la recurrencia. En [23] se destaca que tanto las RNN como las LSTM, aunque eficaces, tenían dos problemas fundamentales: su procesamiento secuencial impedía la paralelización del entrenamiento, y la información de posiciones lejanas tendía a degradarse con la distancia. Estas dificultades se describen en la obra de Y. Bengio [24]. La solución fue el mecanismo de *self-attention*,

que permite a cada token de la secuencia relacionarse directamente con cualquier otro token, independientemente de su posición, en un único paso.

El *Transformer* resultante se organiza en dos bloques: un *encoder*, que construye representaciones contextualizadas de la secuencia de entrada, y un *decoder*, que genera la secuencia de salida token a token apoyándose en dichas representaciones. Mejoró significativamente a las metodologías existentes en traducción automática con un coste computacional significativamente menor y se convirtió en la base de prácticamente todos los modelos de lenguaje modernos. Wayne Xin Zhao hace un recorrido histórico sobre estos aspectos en [3].

3.1.7. Large language models (LLM)

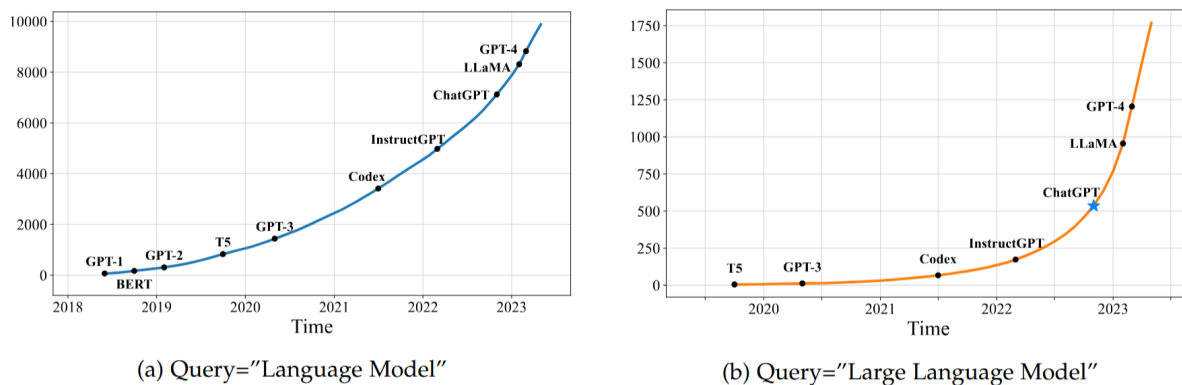


Figura 3.4: Incremento de popularidad LLM. Figura obtenida de [3]

Como podemos ver en el recorrido histórico de Xin Zhao [3] el escalado masivo de los PLM, tanto en número de parámetros como en volumen de datos de entrenamiento, dio lugar a una nueva categoría conocida como *Large Language Models* (LLM). La diferencia con sus predecesores no es solo cuantitativa: al superar ciertos umbrales de escala, estos modelos exhiben capacidades emergentes que los modelos más pequeños no poseen.

En la figura 3.4 observamos como los LLM crecen en popularidad a lo largo del tiempo. Podemos ver claramente como su popularidad aumenta paulatinamente a lo largo de los años hasta llegar a mitades de 2022 donde se dispara con la llegada de ChatGPT.

El ejemplo más representativo es GPT-3 [25], con 175.000 millones de parámetros, capaz de resolver tareas nuevas con apenas unos pocos ejemplos en el propio prompt, conocido como *few-shot learning*, sin necesidad de reentrenamiento. Esta capacidad, denominada *in-context learning*, supone un cambio de paradigma respecto a modelos

anteriores como GPT-2 [26] (1.500 millones de parámetros), que no la poseían de forma efectiva. Otros LLMs de gran escala son PaLM [27], con 540.000 millones de parámetros, o la familia de modelos que da lugar a ChatGPT, cuyo lanzamiento en 2022 popularizó masivamente el uso de estos sistemas entre el público general.

Sin embargo, los LLMs no están exentos de limitaciones. Su conocimiento queda limitado al momento en el que se entrenan, lo que los hace incapaces de incorporar información nueva sin un proceso de reentrenamiento que puede ser costoso.

Además, tienden a generar respuestas plausibles pero incorrectas, fenómeno conocido como *alucinación* descrita en [28], especialmente cuando se les pregunta sobre datos precisos o documentación específica. Estas limitaciones son las que motivaron el desarrollo de la arquitectura RAG, descrita en la siguiente sección.

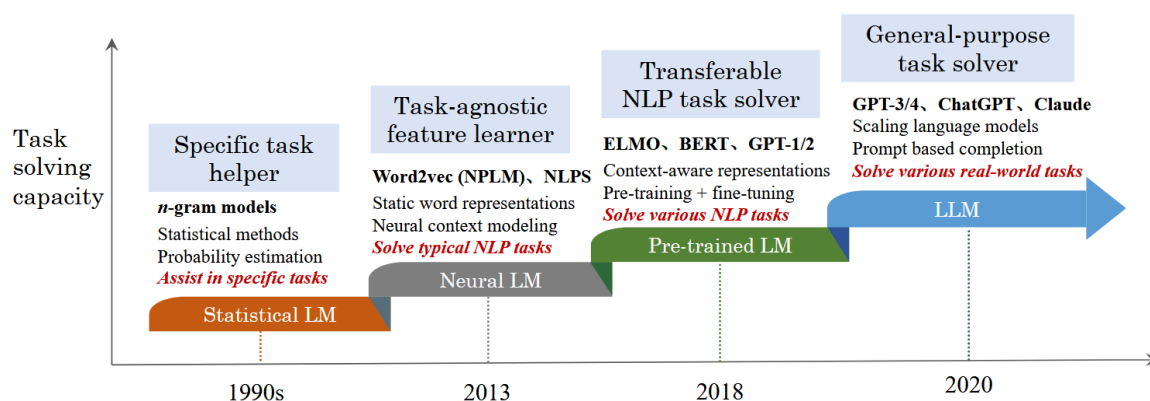


Figura 3.5: Aumento de capacidades LLM. Figura obtenida de [3]

La figura 3.5 hace un recorrido histórico desde 1990 hasta nuestros días de como ha ido aumentando progresivamente la capacidad de los modelos de lenguaje.

3.2. La aparición de RAG

La arquitectura RAG (*Retrieval-Augmented Generation*) fue propuesta por Lewis et al. [29] en 2020 como respuesta directa a las limitaciones descritas anteriormente. La idea central es sencilla: en lugar de que el modelo dependa exclusivamente de su conocimiento interno, se le proporciona acceso a una base de documentos externa en el momento de la consulta.

Se trata de una tecnología muy potente que combina LLMs con documentación actualizada, lo cual nos permite generar respuestas más precisas, actualizadas y relevantes dentro del contexto en el que estemos trabajando. Esta forma de trabajar

resulta especialmente útil para responder preguntas sobre un corpus de documentos propietarios que cambian con frecuencia y que tratan temas específicos.

3.2.1. Componentes clave del RAG

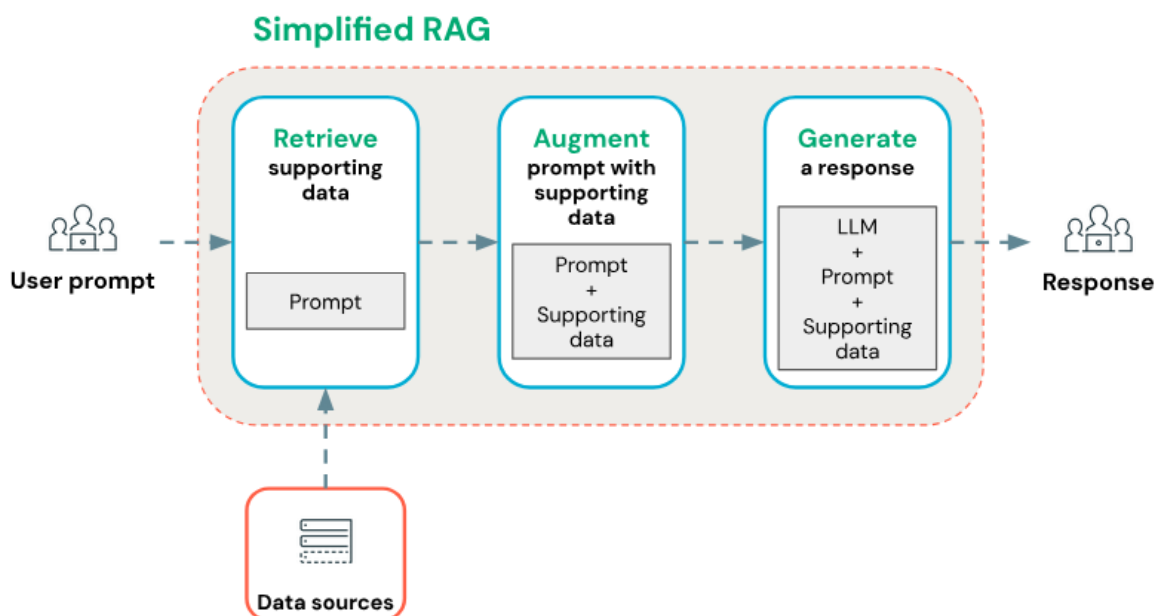


Figura 3.6: Chunking. Figura obtenida de [4]

La figura 3.6 representa de una forma simple las tres partes principales de un sistema RAG:

1. **Recuperación** (*Retrieval*): busca en la base de documentos los fragmentos más relevantes para la pregunta del usuario.
2. **Aumento** (*Augmentation*): esos fragmentos se combinan con la solicitud del usuario, normalmente mediante una plantilla con formato adicional e instrucciones para el LLM, formando así un prompt.
3. **Generación** (*Generation*): ese prompt se pasa al LLM para que genere una respuesta a la solicitud del usuario.

3.2.2. Beneficios de RAG

Como podemos ver en [29], los sistemas RAG mejoran a los LLM tradicionales en los siguientes puntos:

- Conocimiento propietario:** Podemos incluir en la base de conocimiento del LLM sin la necesidad de un reentrenamiento.

- Información actualizada:** Permite actualizar el conocimiento del sistema simplemente modificando la base documental.

- Trazabilidad:** hace las respuestas trazables hasta sus fuentes originales.

Estas ventajas lo convierten en una arquitectura especialmente adecuada para aplicaciones donde la precisión y la fiabilidad son críticas, como es el caso del asistente académico que se desarrolla en este trabajo.

3.2.3. Aplicaciones en el ámbito educativo

En el entorno educativo, los sistemas RAG permiten desarrollar sistemas de tutoría inteligente basados en documentación de las propias asignaturas [30]. Al restringir la base documental a materiales académicos concretos, el sistema puede proporcionar respuestas alineadas con el contenido del curso, ofrecer explicaciones fundamentadas en textos oficiales y generar retroalimentación personalizada. Como comentamos en 3.2.2 uno de los grandes beneficios del RAG es la trazabilidad y el control sobre las fuentes. Esto convierte al RAG en una herramienta particularmente adecuada para contextos formativos, que es precisamente el caso de uso que aborda este trabajo.

Capítulo 4

DISEÑO DE UN SISTEMA RAG

Esta parte la dedicaremos a la abstracción de cada una de las partes del proyecto. En cada una de ellas realizaremos una explicación general de su propósito y como lo abordaremos en el capítulo 5: Materiales y métodos.

4.1. Cadena para la gestión de la documentación no estructurada (*Unstructured data pipeline*)

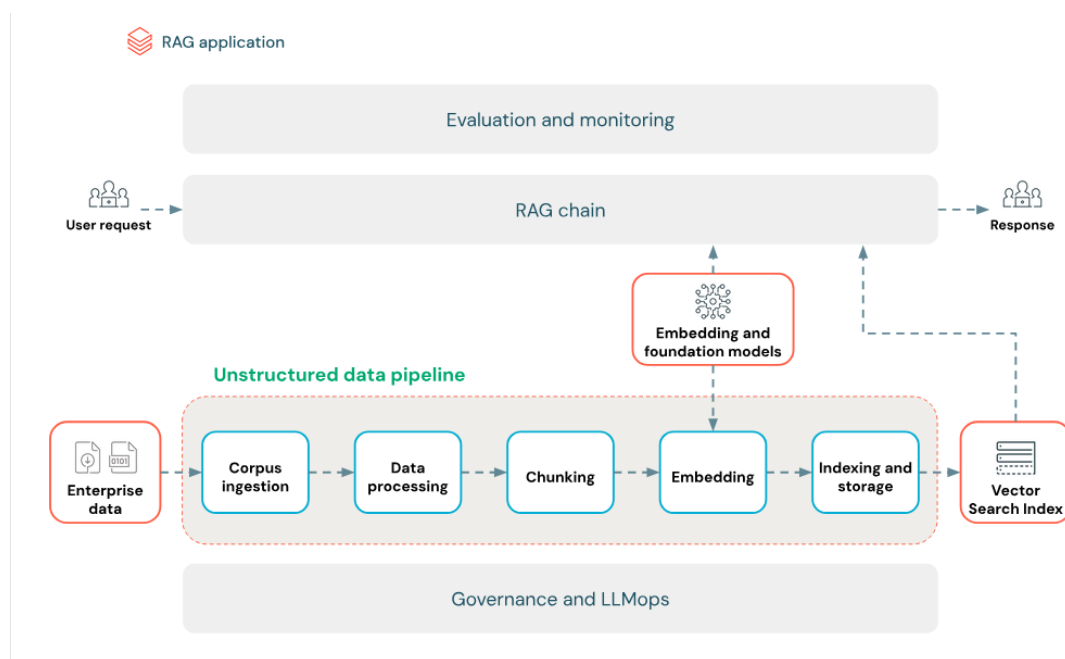


Figura 4.1: Unstructured data pipeline. Figura obtenida de [5]

Usaremos la web de *Databricks* [5] para estructurar y poder explicar adecuada-

mente esta parte.

4.1.1. Selección e ingestión del corpus

Para nuestro asistente virtual utilizaremos la documentación de la asignatura de sistemas operativos de la Universidad de Jaén. Al ser documentos técnicos muy diferentes estructuralmente entre sí es una tarea que necesita un análisis minucioso. Algunos de los elementos que componen el corpus y los cuales deberemos decidir como gestionar son los siguientes: Tablas en diferentes formatos, encabezados, imágenes y diagramas entre otros.

4.1.2. Procesamiento de la documentación

Tendremos que transformar la documentación en formato PDF a formato mark-down.

La documentación inicialmente nos ha sido entregada en formato PDF, tendremos que limpiar y transformar estos documentos a formato Markdown, una representación que preserva la jerarquía y estructura del contenido de forma adecuada para su posterior ingestión por el sistema. Gracias a este formato nuestro sistema RAG será capaz de leer el corpus de documentos de la forma más completa posible.

Análisis sintáctico de los documentos (*Parsing*)

Después de seleccionar la documentación tendremos utilizar una herramienta para extraer la información que necesitamos. Este proceso se denomina *parsing*. Para evitar que nuestro sistema utilice datos basura tendremos que hacer un buen preprocesamiento en el que nos deshagamos del ruido que aportarían los encabezados, pies de página, índices y demás texto plano que se repite de manera sistemática en todos los documentos.

4.1.3. *Chunking*

Consiste en trocear el texto ya extraído para obtener pedazos más pequeños y manejables como podemos ver en la figura 4.2. Esto permite que el contenido recuperado quepa en la ventana de contexto de nuestro LLM además de minimizar la cantidad de información irrelevante que incluimos.

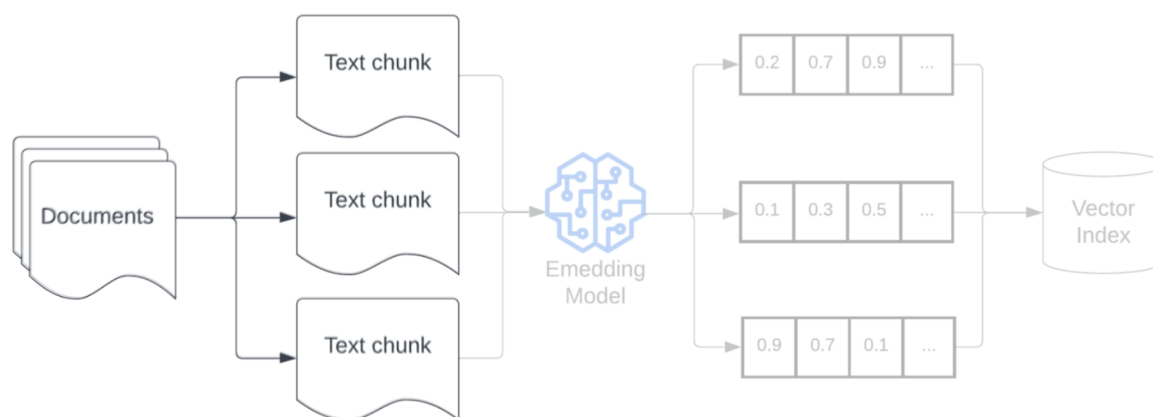


Figura 4.2: Chunking. Figura obtenida de [5]

La forma en la que se realice el *Chunking* afectará de forma directa en la precisión de la información recuperada, haciendo que esta sea una de las primeras capas que determinarán la calidad final de una aplicación RAG de manera considerable.

Para afrontar el *Chunking* de nuestro proyecto deberemos tener en cuenta los siguientes factores:

- Estrategia de *chunking*:** La forma en la que dividamos el texto original en chunks. Se profundizará más sobre como crear una estrategia de chunking sólida más adelante 4.1.3.

- Tamaño de los *chunks*:** El uso de *chunks* pequeños favorece que se centre más la atención sobre detalles específicos. Por otro lado, *chunks* de mayor tamaño son capaces de aportar mayor contexto a la vez que incluir más cantidad de información irrelevante.

- Sobrepaginado:** Para minimizar la pérdida de información importante cuando realicemos la división de los *chunks* deberemos utilizar el sobrepaginado. Esta técnica nos ayuda a preservar la continuidad y el contexto a lo largo de todos los *chunks* para mejorar la calidad en la fase de recuperación.

Supongamos que nuestro sistema trocea los documentos en *chunks* de 300 tokens.

Puede darse el caso de forma frecuente en el que el LLM recupere un *chunk* cuya información relevante ha quedado dividida entre el final del *chunk* devuelto por el LLM y el siguiente *chunk*. Mediante el uso de la técnica de paginación podemos añadir al *chunk* devuelto una parte del siguiente, por ejemplo los primeros 50 tokens.

De esta forma prevenimos de cierta manera que el contexto relevante que queremos obtener de un *chunk* quede dividido en dos y parte de este se pierda en la recuperación. Podremos ver de forma visual como funciona esta técnica más adelante en la figura 4.3.

Estrategias de *chunking*

Dar con el método de *chunking* adecuado es un proceso dependiente del contexto. No existe un tamaño de *chunk* que sea el adecuado para todos los proyectos. El tamaño de *chunk* óptimo dependerá en el caso de uso específico y en el corpus de documentos que se procesan. A grandes rasgos, podemos mencionar las siguientes estrategias:

Tamaño de *chunks* fijo: Se divide el texto en chunks de un tamaño predeterminado, como un número fijo de caracteres o tokens. Aunque dividir por un número arbitrario de caracteres o tokens resulta ser la forma más rápida y sencilla, normalmente no da lugar a chunks semánticamente coherentes de forma consistente.

A continuación tenemos un ejemplo de uso de esta estrategia de chunking sobre un fragmento del corpus de documentos de nuestro proyecto.

ChunkViz [31] ofrece una forma interactiva de visualizar cómo distintos valores de tamaño de chunk y de sobrepaginación afectan a los chunks resultantes.

En este caso se usa *RecursiveCharacterTextSplitter*[32] de *LangChain*, con `chunkSize=451` y `chunkOverlap=100`. Cada uno de los *chunks* se representan de un color diferente a excepción del distintivo de los 100 tokens de sobrepaginado que se representan en verde oscuro.

***Chunking* basado en párrafos:** utiliza los límites naturales de los párrafos del texto para definir los chunks. Este método ayuda a preservar la coherencia semántica de los chunks, ya que los párrafos suelen contener información relacionada.

***Chunking* específico por formato:** formatos como Markdown o HTML tienen una estructura inherente que puede definir los límites de los chunks (por ejemplo, los encabezados de Markdown).

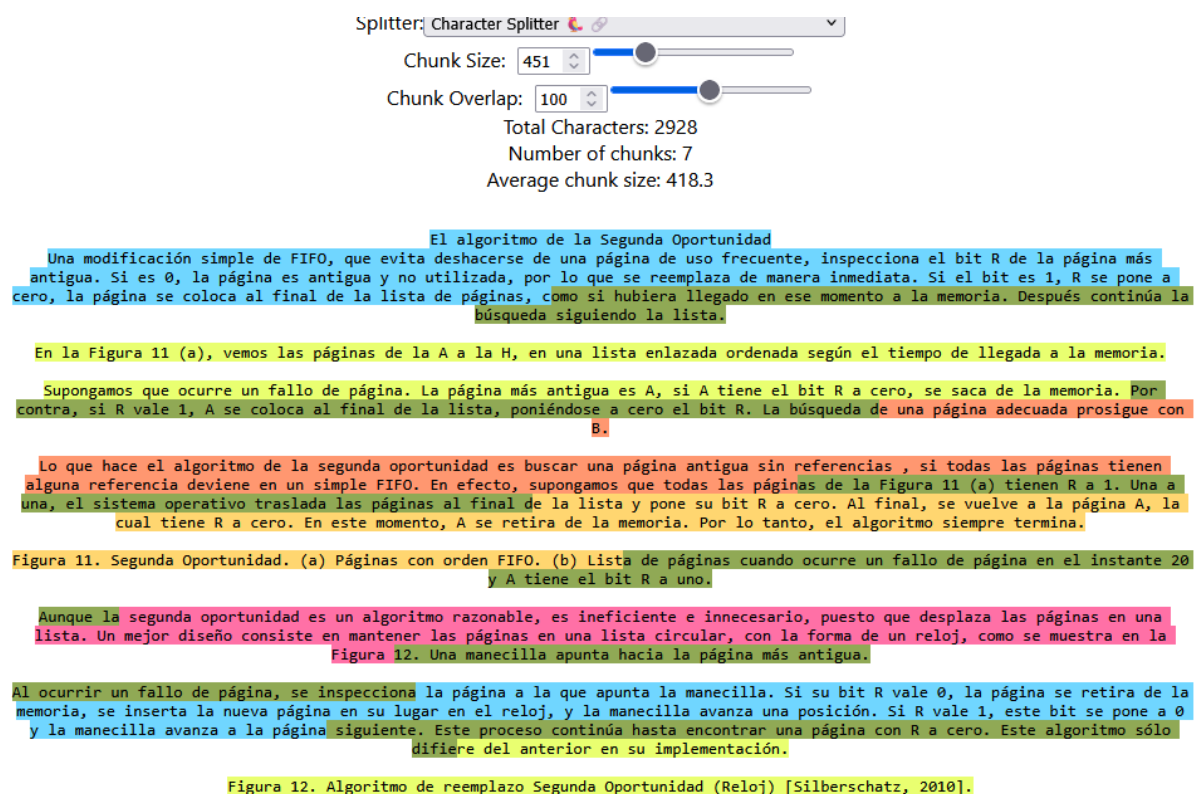


Figura 4.3: Chunks de tamaño fijo. Figura obtenida de <https://chunkviz.up.railway.app/>

Chunking semántico: pueden aplicarse técnicas como el modelado de temas (*topic modeling*) para identificar secciones semánticamente coherentes en el texto. Este enfoque analiza el contenido o la estructura de cada documento para determinar los límites de *chunk* más adecuados en función de los cambios de tema. Aunque resulta más laborioso que otros enfoques más básicos, el chunking semántico ayuda a crear chunks más alineados con las divisiones semánticas naturales del texto.

4.1.4. Embedding

Tras dividir los documentos en *chunks*, el siguiente paso es convertir esos *chunks* de texto en una representación vectorial mediante un modelo de embeddings. Un modelo de embeddings convierte cada chunk de texto en una representación vectorial que plasma su significado semántico. Al representar los chunks como vectores, los embeddings permiten recuperar de forma rápida y precisa los chunks más relevantes en función de su similitud semántica con la consulta que realice un usuario. Esa consulta se transforma en el momento de la búsqueda utilizando el mismo modelo de embeddings que se empleó para generar los embeddings de los chunks en el pipeline de datos.

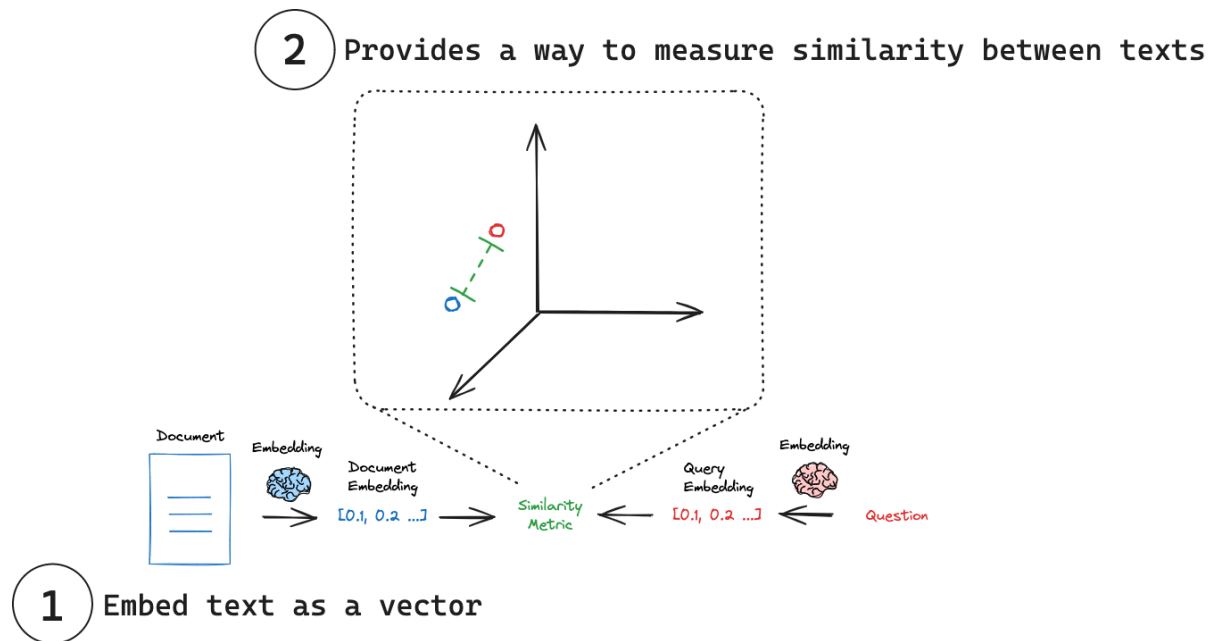


Figura 4.4: *Comparación consulta-chunks*. Figura obtenida de [6]

La figura 4.4 permite visualizar como se compara la distancia entre los embeddings de la consulta y los embeddings de los chunks en un espacio tridimensional.

¿Cómo elegir un modelo de embeddings?

A la hora de elegir un modelo de embeddings, conviene tener en cuenta los siguientes factores:

-Elección del modelo: cada modelo de embeddings tiene sus propios matices, y los benchmarks pueden no reflejar las características específicas de un corpus específico. Es fundamental elegir un modelo entrenado con datos similares a los que vayamos a usar en nuestro proyecto. También puede ser útil explorar modelos de embeddings diseñados para tareas concretas. Experimenta con distintos modelos ya disponibles, incluso con aquellos que aparezcan peor posicionados en leaderboards estándar como MTEB [33].

-Límite máximo de tokens: es importante conocer el límite máximo de tokens de nuestro modelo de embeddings. Si le pasamos chunks que superen ese límite, se truncarán, con la posible pérdida de información relevante pese a que podamos llegar a amortiguar de cierta manera esta pérdida con el sobrepaginado.

-Tamaño del modelo: los modelos de embeddings más grandes suelen rendir mejor, pero requieren más recursos computacionales. En función de nuestro caso de uso

concreto y de los recursos disponibles tenemos que encontrar el equilibrio entre rendimiento y eficiencia.

Indexado y almacenamiento

Como se explica en la web de *DataBricks* [34]. Esta etapa consiste en organizar los embeddings en estructuras de datos eficientes que permitan búsquedas de similitud rápidas y precisas. Para ello usaremos bases de datos vectoriales.

Una base de datos vectorial es una base de datos diseñada para almacenar y gestionar datos en forma de vectores de alta dimensionalidad. A diferencia de las bases de datos tradicionales, la estructura de una base de datos vectorial organiza la información como representaciones vectoriales con un número fijo de dimensiones, agrupadas según su similitud.

Cada vector dentro de una base de datos vectorial consta de un número determinado de dimensiones, que puede oscilar entre unas pocas decenas y varios miles. Ese número de dimensiones depende de la complejidad y la granularidad de los datos. Esta estructura permite a las bases de datos vectoriales manejar de forma eficiente información compleja, y realizar búsquedas y análisis basados en similitud de manera rápida.

En la figura 3.3 podemos observar una base de datos vectorial representada tridimensionalmente.

4.2. Cadena RAG para la inferencia (*RAG chain for inference*)

Una vez que la documentación esté troceada, vectorizada e indexada, tendremos que utilizarla para responder a las preguntas de los estudiantes. Al igual que en la sección anterior, usaremos la web de *Databricks* para estructurar y poder explicar adecuadamente esta parte, en concreto sus páginas sobre la cadena RAG [35] y sobre cómo mejorar su calidad [36].

Se denomina cadena RAG (*RAG chain*) a la serie de pasos que se ejecutan cada vez que un usuario realiza una consulta. A diferencia de la cadena de la sección anterior, que se ejecuta una sola vez o cada vez que cambia la documentación, esta cadena se ejecuta en tiempo real, por lo que tendrá que responder en pocos segundos.

En los Antecedentes vimos las tres partes principales de un sistema RAG (figura 3.6): recuperación, aumento y generación. La cadena RAG añade a estas tres un paso previo y otro posterior, ambos opcionales:

1. **Preprocesado de la consulta** (*Query understanding*): prepara la pregunta del usuario antes de buscar.
2. **Recuperación** (*Retrieval*): busca los *chunks* más relevantes para la pregunta.
3. **Aumento del *prompt*** (*Prompt augmentation*): combina la pregunta con los *chunks* recuperados en una única instrucción para el LLM.
4. **Generación** (*LLM*): el LLM redacta la respuesta a partir de esa instrucción.
5. **Post-procesado** (*Post-processing*): revisa la respuesta antes de mostrarla al usuario.

Decisiones aparentemente menores, como cuántos *chunks* recuperar o cómo combinarlos con la pregunta, pueden afectar de forma significativa a la calidad de las respuestas. A continuación veremos en qué consiste cada uno de estos pasos y los desafíos que plantea en nuestro proyecto.

4.2.1. Preprocesado de la consulta (*Query understanding*)

La pregunta que escribe el usuario no siempre es la mejor consulta posible para buscar en la base de datos vectorial. El caso más claro se da en una conversación: supongamos que un estudiante pregunta cómo funciona el algoritmo de reemplazo de páginas del reloj y a continuación escribe «¿y en LRU?». Esa segunda pregunta no se parece a nada del corpus si se busca tal cual, ya que le falta el contexto de la anterior.

Podemos resolverlo con las siguientes dos técnicas:

Reescritura de la consulta: se utiliza un LLM para reformular la pregunta de forma que aumente la probabilidad de encontrar los documentos adecuados, por ejemplo resolviendo las referencias a mensajes anteriores de la conversación, corrigiendo errores ortográficos o sustituyendo palabras por sinónimos.

Extracción de filtros: se identifican en la pregunta restricciones concretas, como un tema o un tipo de documento, que permitan acotar la búsqueda.

Tendremos que decidir si merece la pena añadir este paso, en qué parte de la conversación necesitamos que el modelo que reescribe la pregunta actúe y cómo

evitar que esta llamada adicional afecte al ritmo de la conversación.

4.2.2. Recuperación (*Retrieval*)

Como explicamos en la sección 4.1.4, la consulta se transforma en un *embedding* con el mismo modelo que se empleó para los *chunks*, y se buscan en la base de datos vectorial aquellos cuya similitud con ella sea mayor. Esta es la etapa que decide qué información va a leer el LLM y, por tanto, una de las que más influyen en la calidad final: si el *chunk* que contiene la respuesta no se recupera, ningún paso posterior podrá compensarlo.

Para afrontar la recuperación de nuestro proyecto deberemos tener en cuenta los siguientes factores:

-**Terminología exacta frente a paráfrasis:** el temario está lleno de términos técnicos exactos, como el nombre de una orden de *shell* o siglas como FIFO o LRU, que la búsqueda por similitud semántica puede pasar por alto. Sin embargo, los estudiantes también formularán sus preguntas con sus propias palabras, que no siempre coincidirán con las de los apuntes.

-**Número de *chunks* recuperados:** al igual que ocurría con el tamaño de los *chunks*, recuperar pocos puede dejar fuera información necesaria, sobre todo en preguntas que obligan a combinar varios temas, mientras que recuperar demasiados introduce información irrelevante en el contexto y encarece cada consulta.

-**Orden y redundancia:** el orden en el que llegan los *chunks* al LLM importa. Además, el sobrepaginado que describimos en la sección 4.1.3 hace que *chunks* contiguos compartan parte de su texto, por lo que pueden ocupar varias posiciones del contexto repitiendo la misma idea.

Estrategias de recuperación

Al igual que ocurría con el *chunking*, no existe una estrategia de recuperación que sea la adecuada para todos los proyectos. La guía [36] compara las siguientes:

Búsqueda semántica: compara los *embeddings* de la consulta y de los *chunks*, por lo que encuentra documentos conceptualmente similares aunque no compartan palabras. Es la opción adecuada cuando el usuario formula la pregunta de forma distinta a como está escrita la documentación.

Búsqueda por palabras clave: puntúa la relevancia de cada documento a partir de la frecuencia y la distribución de las palabras de la consulta, con técnicas como TF-IDF. Es la opción adecuada cuando importa la coincidencia exacta de un término.

Búsqueda híbrida: combina las dos anteriores y fusiona sus resultados, para aprovechar las ventajas de ambas.

Sobre cualquiera de estas estrategias pueden aplicarse además técnicas que mejoran el resultado: la **expansión de la consulta**, que busca con varias variantes de la misma pregunta; el **reordenado** (*re-ranking*), que aplica un modelo más preciso sobre los primeros resultados para ordenarlos mejor, y el **filtrado por metadatos**, que acota la búsqueda a los documentos que cumplen ciertas condiciones.

4.2.3. Aumento del *prompt* (*Prompt augmentation*)

Una vez recuperados los *chunks*, se construye la instrucción (*prompt*) que recibirá el LLM combinando la pregunta del usuario con el contexto recuperado mediante una plantilla. Esa plantilla indica al modelo cómo debe usar cada parte y qué formato debe tener la respuesta.

En nuestro caso, la plantilla será la principal herramienta para aprovechar la trazabilidad que comentamos en 3.2.2: tendremos que conseguir que el modelo indique de qué *chunk* sale cada afirmación, que se limite a la documentación de la asignatura y que reconozca cuándo la información no está disponible en lugar de inventarla.

4.2.4. Generación (LLM)

El LLM recibe el *prompt* aumentado y redacta la respuesta. Lo ideal sería experimentar con distintos modelos, empezar por modelos ligeros y escalar solo si es necesario, utilizar modelos diferentes para pasos diferentes de la cadena y ajustar parámetros como la temperatura, que controla la aleatoriedad de las respuestas, o su longitud máxima.

El principal riesgo de esta etapa es la alucinación que describimos en los Antecedentes 3.1.7: un modelo que, al no encontrar la respuesta en el contexto, la completa con información plausible pero incorrecta o ajena a la asignatura. En un asistente académico este riesgo es especialmente grave, ya que la información que aprende el estudiante puede ser errónea sin tener forma de detectarlo.

¿Cómo elegir un modelo de lenguaje?

Al igual que ocurría con los modelos de embeddings, a la hora de elegir el LLM que redactará las respuestas conviene tener en cuenta los siguientes factores:

-**Calidad y seguimiento de instrucciones:** el modelo debe redactar con corrección en español y, sobre todo, respetar las reglas del *prompt*: citar las fuentes y no salirse del contexto.

-**Ventana de contexto:** el número de *tokens* que el modelo es capaz de procesar limita cuánta información recuperada podemos incluir en cada consulta.

-**Coste:** los modelos que se consumen a través de una API se facturan por *token* de entrada y de salida, mientras que los que se ejecutan en local requieren un *hardware* considerable.

-**Latencia:** deberemos reducir el tiempo de espera entre respuesta y respuesta lo máximo posible.

-**Reproducibilidad:** para poder evaluar el sistema, la misma pregunta con el mismo contexto debería producir la misma respuesta, y la versión del modelo no debería cambiar a mitad de la experimentación.

4.2.5. Post-procesado

Este último paso garantiza que la respuesta cumple con los requisitos de formato y estilo antes de llegar al usuario. Se deberá comprobar el formato de salida, la coherencia de estilo, la ausencia de alucinaciones contrastando la respuesta con los documentos recuperados y la gestión de errores mediante respuestas por defecto en caso de que no se conozca la respuesta. En nuestro caso tendremos que comprobar que las citas del modelo corresponden a *chunks* que realmente ha recibido y detectar cuándo el sistema ha decidido no responder.

4.3. Evaluación y monitorización (*Evaluation and monitoring*)

Una vez construido el sistema tendremos que medir si responde bien. Evaluar un sistema RAG no es tan sencillo como comparar su salida con una respuesta esperada: las respuestas de un LLM no son deterministas y una misma pregunta admite muchas respuestas correctas redactadas de forma distinta. Podemos medir las respuestas en tres dimensiones: la **calidad** de sus respuestas, su **coste** y su **latencia**.

4.3.1. Los dos tipos de calidad

Para mejorar un sistema RAG hay que saber primero dónde falla. Existen dos tipos de calidad, que conviene diagnosticar por separado porque cada uno se corrige de forma diferente:

Calidad de la recuperación: ¿estamos recuperando la información más relevante para la pregunta? Depende sobre todo de los elementos de la cadena de datos que vimos en la sección 4.1. La composición del corpus, el *parsing*, el *chunking*, los metadatos de cada *chunk* y el modelo de embeddings, además de la estrategia de recuperación.

Calidad de la generación: ¿está generando el LLM la respuesta más precisa, coherente y útil posible con el contexto que ha recibido? Depende sobre todo de los elementos de la cadena RAG que acabamos de ver: la elección y configuración del LLM, el número de *chunks* recuperados y el formato del *prompt*.

4.3.2. Evaluación

Antes de poder medir la calidad hay que definirla, y para ello se necesita un conjunto de evaluación. Podríamos usar una colección seleccionada de preguntas acompañadas de su respuesta correcta (*ground truth*) y, opcionalmente, de los documentos que deberían recuperarse para contestarlas.

Un buen conjunto de evaluación debe cumplir las siguientes características:

-**Representativo:** tiene que reflejar la variedad de preguntas que la aplicación recibirá realmente. En nuestro caso, las de un estudiante de Sistemas Operativos.

-**Exigente**: tiene que incluir casos difíciles y diversos. Como establecimos en los objetivos, las preguntas deberán cubrir distintos niveles de complejidad, desde preguntas de definición hasta preguntas de razonamiento.

-**Actualizado**: tiene que revisarse a medida que cambian el uso del sistema y la documentación.

4.3.3. Evaluar el rendimiento: métricas

Con el conjunto de evaluación definido, tendremos que decidir qué medir. La guía [37] organiza las métricas en tres grupos:

-**Calidad de la recuperación**: si el sistema recupera la información relevante. Se basa en dos conceptos clásicos: la **precisión**, que mide qué proporción de los *chunks* recuperados es relevante y penaliza traer ruido al contexto, y la **exhaustividad** (*recall*), que mide qué proporción de la información relevante se ha recuperado y penaliza dejar fuera información necesaria. A ellas se suma la suficiencia del contexto, es decir, si lo recuperado basta para responder.

-**Calidad de la respuesta**: si la respuesta es correcta, si es relevante para la pregunta, si está fundamentada en el contexto recuperado (*groundedness*) y si es segura.

-**Rendimiento**: el coste, medido en *tokens* de entrada y de salida, y la latencia.

Estas métricas pueden obtenerse de dos formas: de manera **determinista**, como el número de *tokens* o el tiempo de respuesta, o mediante un **juez LLM**, es decir, un modelo de lenguaje adicional que valora aspectos que no pueden reducirse a una fórmula, como si una afirmación está sustentada por el contexto. Además, algunas métricas necesitan la respuesta de referencia del conjunto de evaluación, como la corrección. Tendremos que elegir una herramienta que calcule métricas de los dos tipos de calidad adaptadas a nuestro caso y un modelo que actúe como juez.

Capítulo 5

MATERIALES Y MÉTODOS

En el anterior capítulo [4](#) describimos cada una de las partes de un sistema RAG y los desafíos que plantean para nuestro proyecto. En este capítulo realizaremos una investigación sobre qué tecnologías se utilizan en cada una de esas partes. Para cada una de ellas presentaremos un conjunto de alternativas, las compararemos brevemente y nos decantaremos por una, explicando por qué es la más adecuada para nuestro caso de uso.

Seguiremos el mismo orden que en el capítulo del diseño RAG: primero el entorno de trabajo, después la cadena de datos que prepara la documentación, a continuación la cadena RAG que responde a las preguntas y, por último, la evaluación del sistema. La figura [5.1](#) muestra este recorrido de forma esquemática.

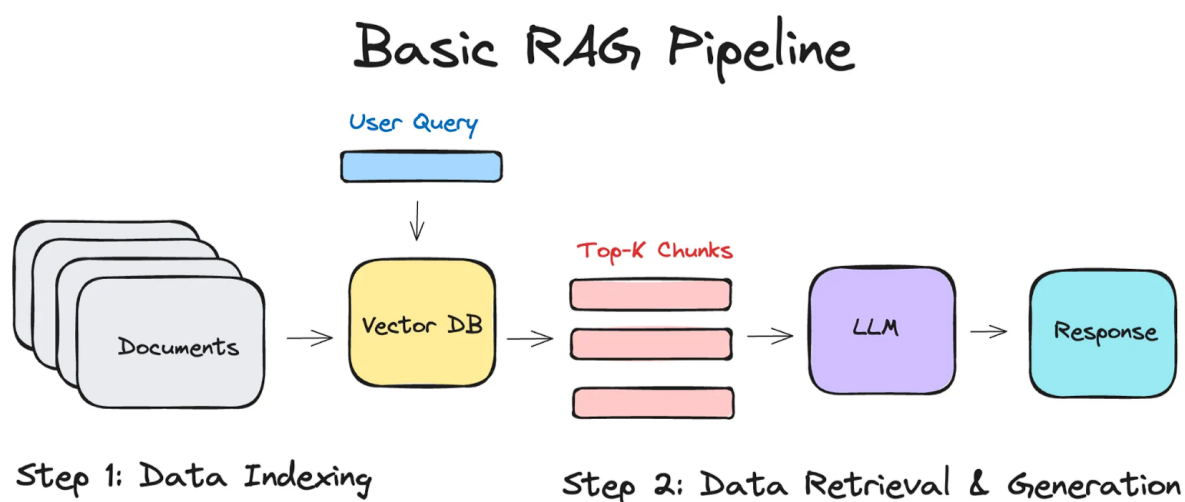


Figura 5.1: Pipeline básico RAG. Figura obtenida de [\[7\]](#)

Durante el proyecto se priorizarán tecnologías gratuitas para el desarrollo. Existen excepciones como el uso de un modelo de OpenAI que se usará para realizar compa-

rativas pero ese modelo no forma parte del sistema final.

Las decisiones que se tomaron midiendo el sistema se justifican aquí de forma cualitativa. Las mediciones en las que se apoyan se presentan en el capítulo de Resultados (capítulo 6).

5.1. Entorno y herramientas

5.1.1. Lenguaje de programación y entorno de ejecución

Todo el desarrollo se ha realizado en Python, el lenguaje estándar en el ámbito de la inteligencia artificial. Esto se traduce en una mayor cantidad de librerías, ejemplos e integraciones disponibles, y en más facilidades para resolver los problemas que surgen durante el desarrollo.

En concreto se ha utilizado la versión 3.12 [38]. Es una versión madura, con la que todas las librerías del proyecto son compatibles, y evita los problemas de compatibilidad que suelen tener las versiones de publicación más reciente.

Las dependencias se gestionan con `pip` dentro de un entorno virtual y se recogen en un fichero `requirements.txt`, de forma que el entorno pueda reproducirse en cualquier otra máquina. La clave de acceso a la API de los modelos se guarda en un fichero `.env` que no se sube al repositorio, para que la información sensible no quede expuesta en el código fuente.

5.1.2. Framework

Un *framework* para RAG ofrece «piezas» ya hechas (lectores de documentos, divisores de texto, conectores con bases de datos vectoriales y con modelos de lenguaje) que se combinan para montar el sistema sin tener que programarlas desde cero. Estudiamos las siguientes alternativas:

LlamaIndex [39]: diseñado específicamente para construir sistemas RAG. Resulta muy cómodo para montar un prototipo en pocas líneas, a cambio de ocultar buena parte de lo que ocurre por dentro.

Haystack [40]: *framework* de código abierto orientado a construir cadenas de búsqueda y respuesta en entornos de producción. Es sólido, pero cuenta con una comu-

nidad y un número de integraciones menores.

DSpy [41]: propone un enfoque distinto, en el que en lugar de escribir los *prompts* a mano se declaran los pasos del sistema y el propio *framework* los optimiza. Es un enfoque interesante para la investigación, pero añade una capa de complejidad que no necesitamos.

LangChain [42]: *framework* de propósito general para aplicaciones con modelos de lenguaje, del cual el RAG es sólo uno de sus usos.

Nos hemos decantado por **LangChain** por las siguientes razones:

- Tiene la mayor cantidad de recursos disponibles: documentación, tutoriales y ejemplos.
- Ofrece integraciones con todas las piezas que utilizaremos más adelante: la base de datos vectorial, la búsqueda por palabras clave, los modelos servidos por una API compatible con la de OpenAI y la herramienta de evaluación.
- Permite construir cada paso de la cadena por separado. Esto nos permitirá observar cada etapa de forma aislada y saber en qué punto falla el sistema, algo que necesitaremos para la evaluación de la sección 4.3.1.

5.1.3. Proveedor de los modelos

Nuestro sistema necesita varios modelos: uno de embeddings, un LLM que redacte las respuestas y otro LLM que actúe como juez durante la evaluación. Antes de elegir cada modelo hay que decidir de dónde van a salir, ya que esta decisión limita el catálogo disponible. Tenemos tres opciones:

APIs comerciales, como las de OpenAI o Google Gemini [43]: ofrecen los modelos más potentes, pero se pagan por *token*. En una primera versión del prototipo se utilizó Gemini, aprovechando su crédito gratuito, pero la evaluación completa del sistema consume una gran cantidad de peticiones y el crédito se agotó. Esto confirmó que una API de pago no es viable para este proyecto.

Ejecución en local, con herramientas como Ollama [44] o vLLM [45]: es gratuita y no depende de terceros, pero ejecutar un LLM de calidad suficiente requiere de un hardware que no disponemos.

Servicios académicos: Blablador [46] es un servicio del centro de supercompu-

tación de Jülich (Helmholtz, Alemania) que ofrece de forma gratuita LLMs a la comunidad universitaria. Se accede con la cuenta de la universidad y su API es compatible con la de OpenAI, por lo que se integra en LangChain sin código adicional.

Hemos elegido **Blablador**, ya que es la única opción que cumple a la vez con ser gratuita y con ofrecer modelos del tamaño que necesitamos. A cambio, impone un límite de peticiones por minuto, que tendremos en cuenta sobre todo en la evaluación.

La API de OpenAI [47] se utiliza únicamente para el experimento de comparación con un generador de pago. El código admite los dos proveedores para el generador, pero el juez y los embeddings siguen siempre en Blablador, de modo que ese experimento cambia una sola pieza del sistema.

Una precaución importante: Blablador ofrece sus modelos tanto por nombres genéricos (alias) como por su identificador concreto, y los alias pueden pasar a apuntar a otro modelo sin previo aviso. Como veremos en la evaluación, para comparar dos configuraciones del sistema el modelo no puede cambiar entre ellas, por lo que siempre se usa el identificador concreto.

Las únicas piezas que se ejecutan en local son la conversión de los PDF y el modelo de reordenado, ya que son lo bastante ligeras para funcionar sin tarjeta gráfica.

5.2. Cadena para la gestión de la documentación

Esta sección desarrolla la cadena de datos descrita en la sección 4.1: convertir los PDF a Markdown, trocearlos, vectorizarlos y almacenarlos. La figura 5.2 resume este proceso.

5.2.1. Selección del corpus

El corpus está formado por 33 documentos de la asignatura de Sistemas Operativos de la Universidad de Jaén, que podemos agrupar en tres tipos:

- **Apuntes de teoría:** los diez temas de la asignatura, desde los conceptos generales hasta el sistema de ficheros, pasando por procesos, memoria y entrada/salida.
- **Guiones y manuales de prácticas:** sesiones sobre Linux y programación en *shell* Bash.

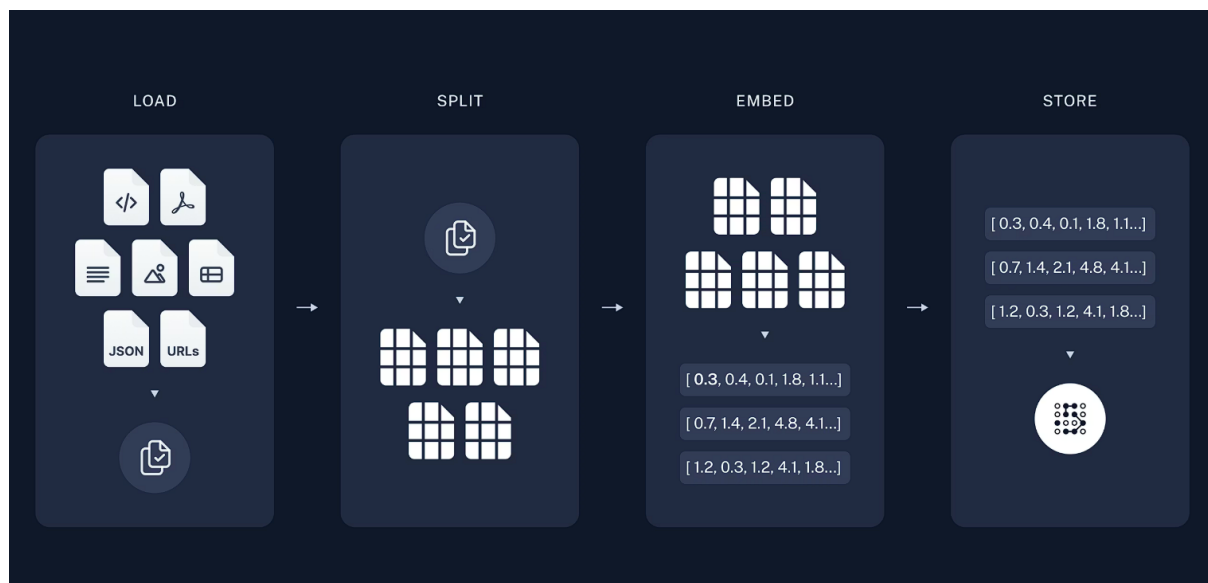


Figura 5.2: Indexación. Figura obtenida de [8]

- **Relaciones de problemas:** ejercicios de los distintos temas y una relación de ejercicios de programación en Bash.

Como adelantamos en el capítulo 4, son documentos muy distintos entre sí: los apuntes tienen prosa, tablas y diagramas; los guiones de prácticas están llenos de órdenes de *shell* y bloques de código, y las relaciones de problemas contienen tablas de datos y cronogramas.

5.2.2. Conversión de PDF a Markdown

El formato PDF está pensado para presentar un documento visualmente, no para describir su estructura: guarda dónde va cada letra y con qué tipografía, pero no qué es un título, qué es una tabla o en qué orden se lee el texto. Nuestro objetivo es recuperar esa estructura en Markdown, ya que, como veremos, es la base de la parte de *chunking*. Por ello, los dos criterios que más valoraremos son, por este orden, **la conservación de los encabezados y la fidelidad de las tablas**.

Las herramientas que existen para esta tarea pueden agruparse en cuatro familias, resumidas en la tabla 5.1:

Extractores de texto: *MuPDF* [48] y su variante para LLMs, *pymupdf4llm*, leen directamente la capa de texto del PDF. Son muy rápidos y ligeros, pero como se muestra en [49] no son capaces de reconstruir tablas complejas con fidelidad. Fue la herramienta utilizada en la primera versión del prototipo y las tablas del temario quedaban

desordenadas.

Herramientas basadas en modelos de *layout*: *Unstructured* [50], *Marker* [51], *MinerU* [52] y *Docling* [53] utilizan modelos de visión que analizan la página como una imagen y etiquetan cada bloque (título, párrafo, tabla, figura. . .). *Unstructured* destaca por la gran variedad de formatos que soporta y mejora mucho a MuPDF en las tablas [54].

Modelos de visión y lenguaje: *Nougat* [55], *olmOCR* [56] o *SmolDocling* [57] leen la imagen de la página y generan directamente el texto. Obtienen buenos resultados, pero el coste de dispa en comparación con las demás alternativas.

Tabla. 5.1: Comparativa de herramientas de conversión de PDF a Markdown

Familia	Tablas y estructura	Coste	Requisitos
Extractores (MuPDF)	Pobre	Gratis	Ninguno
Modelos de <i>layout</i> (Docling. . .)	Buena	Gratis	CPU
Visión y lenguaje (Nougat. . .)	Buena	Gratis	GPU

Hemos elegido **Docling** por las siguientes razones:

- Su modelo de *layout*, entrenado sobre el conjunto DocLayNet [58], identifica los títulos de cada sección y los convierte en encabezados de Markdown. Es la herramienta que mejor ha conservado la jerarquía de nuestros documentos.
- Incorpora *TableFormer* [59], un modelo entrenado específicamente para reconstruir la estructura de las tablas, incluidas las celdas combinadas. Además, toma el texto de cada celda del propio PDF en lugar de predecirlo, por lo que no se inventa contenido.
- Se ejecuta en local sin necesidad de un hardware potente de forma gratuita.

Tras convertir el corpus con las distintas herramientas, pudimos observar a simple vista que MuPDF quedaba muy por detrás de las demás, y que, aunque *Unstructured* hacía un trabajo aceptable, *Docling* conservaba mejor tanto las secciones como las tablas.

Durante la conversión surgieron dos problemas que merece la pena mencionar. El primero es que algunos PDF tienen la capa de texto dañada, de forma que al extraerla aparecen símbolos sin sentido en lugar de letras. En esos casos se descarta la capa de texto y se aplica reconocimiento óptico de caracteres (OCR) sobre la imagen de la página con *RapidOCR* [60], utilizando su modelo para alfabeto latino, ya que el modelo

por defecto no funcionaba correctamente con las tildes y los signos de apertura. El segundo es que Docling no convertía ciertos caracteres especiales de Markdown, lo que añadía ruido a las órdenes de *shell* de los guiones de prácticas.

Procesado incremental

Convertir todo el corpus con Docling lleva un tiempo considerable, y no tiene sentido repetirlo cada vez que se añade un documento nuevo. Para evitarlo se guarda un (*hash* MD5) de cada PDF ya convertido. En cada ejecución se compara el *hash* de cada fichero con la almacenada y solo se convierten los documentos nuevos o modificados.

5.2.3. Limpieza de la documentación

Como vimos en la sección de *parsing* 4.1.2, hay que deshacerse del texto que se repite en todos los documentos y no aporta información. En nuestro corpus el principal ruido proviene de la portada con los datos del curso y un índice.

El índice nos perjudica bastante ya que contiene todos los títulos del tema, por lo que es sencillo que se pueda parecer a casi cualquier pregunta sobre ese tema que el usuario pueda realizar y compite con el contenido real en la búsqueda. Por ello, antes de trocear, se detectan y descartan las portadas y los índices. En algunos casos la introducción del tema aparece pegada a continuación del índice dentro de la misma sección, así que en lugar de descartar la sección completa se conserva esa introducción y se le da un título propio.

5.2.4. *Chunking*

En la sección 4.1.3 vimos cuatro estrategias de *chunking*: tamaño fijo, basada en párrafos, específica por formato y semántica. Existen además propuestas más recientes, como el *late chunking* [61], que vectoriza el documento completo antes de dividirlo, o la recuperación contextual [62], que utiliza un LLM para añadir a cada *chunk* una breve descripción de dónde se encuentra dentro del documento.

Para elegir entre ellas analizamos primero cómo es nuestro corpus una vez convertido a Markdown, y encontramos que **los documentos ya vienen troceados en secciones pequeñas**: la mayoría de las secciones ocupan menos de una página, y

solo una pequeña parte son realmente largas. Dividir por un tamaño fijo, como proponen las recomendaciones habituales, partiría secciones por la mitad o juntaría en un mismo *chunk* varias secciones que no tienen nada que ver entre sí.

Por ello se ha optado por un **chunking específico por formato**, que utiliza los encabezados de Markdown como límites naturales y respeta la sección como unidad. Sobre esta base solo se corrigen sus defectos:

- **Secciones demasiado pequeñas:** las secciones contiguas muy cortas se fusionan hasta alcanzar unos 1200 caracteres, siempre dentro del mismo documento y del mismo tema. Una sección de dos líneas, por sí sola, apenas aporta información.
- **Secciones demasiado grandes:** solo las que superan los 2500 caracteres se subdividen, intentando cortar por párrafos antes que por líneas, y por líneas antes que por palabras. Únicamente en este caso se aplica el sobrepaginado descrito en la sección 4.1.3, de 375 caracteres. Si al cortar queda partido un bloque de código o una tabla, se repara para que ambos trozos sigan siendo legibles.
- **Falta de contexto:** a cada *chunk* se le antepone el título del documento y el de su sección, por ejemplo «05 Memoria. Gestión de Memoria → 6.3. Paginación». Palabras como «página», «planificación» o «proceso» aparecen en temas muy distintos de la asignatura, y sin esa información un *chunk* resulta ambiguo.

Descartamos el *chunking* semántico porque su coste computacional adicional no siempre se traduce en mejores resultados [63], y en nuestro caso la propia estructura del documento ya separa los temas. La recuperación contextual consigue un efecto parecido al de anteponer los encabezados, pero necesita una llamada a un LLM por cada *chunk*. El resultado final es un total de 762 *chunks*, la gran mayoría de los cuales corresponden a una sección completa, con sus bloques de código y sus tablas sin cortar.

5.2.5. Embeddings

En la sección 4.1.4 establecimos tres criterios para elegir un modelo de embeddings: que se adapte a nuestros datos, su número máximo de *tokens* y su tamaño. En nuestro caso, adaptarse a los datos significa sobre todo **funcionar bien en español**. Para comparar los candidatos nos apoyamos en MMTEB [64], la versión multilingüe del *benchmark* MTEB [65] que vimos en el capítulo 4.

all-MiniLM-L6-v2 [66, 67]: modelo pequeño que se ejecuta en local. Fue el utilizado en la primera versión del prototipo, pero está entrenado principalmente en inglés y solo lee unos 256 *tokens*, por lo que truncaba buena parte de nuestros *chunks*.

multilingual-E5 [68] y **BGE-M3** [69]: modelos multilingües abiertos de tamaño medio, con buenos resultados en español. BGE-M3, en concreto, admite textos de hasta 8192 *tokens*.

OpenAI text-embedding-3 [70] y **Gemini Embedding** [71]: modelos comerciales de muy buena calidad, pero de pago. Gemini Embedding se utilizó en la etapa en la que el sistema funcionaba con Gemini.

Qwen3-Embedding-8B [72]: modelo abierto de gran tamaño que se encuentra entre los primeros puestos de MMTEB, admite textos largos y está disponible de forma gratuita en Blablador.

Hemos elegido **Qwen3-Embedding-8B** porque es el que mejor combina los tres criterios: es de los mejores modelos multilingües disponibles, lee sin problemas nuestros *chunks* más largos y, podemos disponer de él gracias a Blablador de forma gratuita.

Este modelo distingue entre el texto que se guarda y la pregunta que lo busca: a las preguntas se les antepone una breve instrucción que describe la tarea («dada una pregunta, recupera los fragmentos que la responden»), mientras que los *chunks* se vectorizan tal cual. Además, como ya advertimos en el capítulo 4, el mismo modelo debe usarse tanto al indexar como al consultar. Si no fuera así, el sistema no daría ningún error: simplemente devolvería resultados sin sentido. Por ello, la configuración del modelo se define en un único lugar que utilizan tanto el indexado como la consulta.

5.2.6. Base de datos vectorial

Para almacenar los vectores y buscar los más parecidos a una consulta estudiamos las alternativas de la tabla 5.2:

FAISS [73]: la referencia en velocidad de búsqueda por similitud, pero es una librería y no una base de datos: no guarda los datos en disco ni gestiona metadatos, por lo que tendríamos que construir esa capa nosotros.

Qdrant [74], **Weaviate** [75] y **Milvus** [76]: bases de datos vectoriales completas pensadas para grandes volúmenes de datos. Normalmente se despliegan como un

servidor independiente, lo cual es excesivo para nuestro proyecto.

Pinecone [77]: servicio en la nube totalmente gestionado, cómodo de usar pero de pago a partir de cierto uso.

pgvector [78]: extensión de PostgreSQL, muy útil cuando ya se dispone de esta base de datos, que no es nuestro caso.

ChromaDB [79]: base de datos vectorial de código abierto que se ejecuta dentro del propio programa y guarda los datos en una carpeta local.

Tabla. 5.2: Comparativa de bases de datos vectoriales

Tecnología	Persistencia	Despliegue	Coste
FAISS	No	Librería	Gratis
Qdrant, Weaviate, Milvus	Sí	Servidor	Gratis
Pinecone	Sí	Nube	De pago
pgvector	Sí	Servidor PostgreSQL	Gratis
ChromaDB	Sí	Integrada, local	Gratis

Hemos elegido **ChromaDB** por las siguientes razones:

- Es la de menor dificultad de despliegue: no necesita ningún servidor.
- Tiene integración nativa con LangChain, el *framework* elegido en la sección 5.1.2.
- Con 762 *chunks*, las ventajas de rendimiento de las alternativas más grandes no tendría ningún efecto apreciable.

Internamente, ChromaDB organiza los vectores en un grafo HNSW [80], que permite encontrar los más cercanos a una consulta sin compararla con todos. Como medida de similitud se utiliza la distancia del coseno. Junto a cada vector se guarda el documento del que procede, su tema, su sección y un identificador único, que es lo que permite mostrar al usuario de dónde sale cada respuesta.

5.3. Cadena RAG para la inferencia

Esta sección desarrolla los cinco pasos de la cadena RAG descritos en la sección 4.2. La figura 5.3 muestra de forma esquemática la recuperación y la generación, que son los pasos centrales.

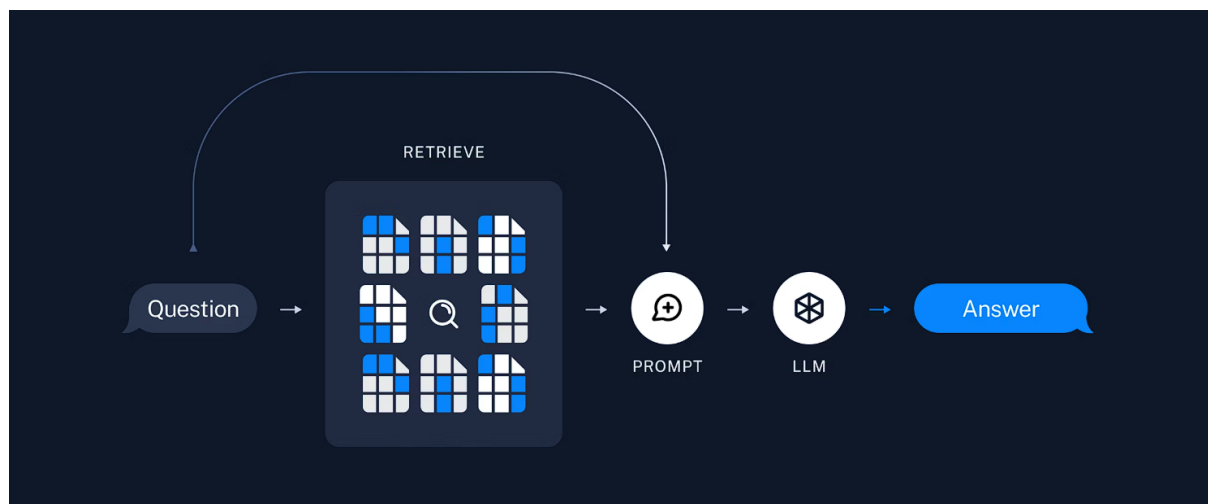


Figura 5.3: Recuperación y generación de respuesta. Figura obtenida de [9]

5.3.1. Preprocesado de la consulta

En la sección 4.2.1 vimos el problema de las preguntas de seguimiento, como «¿y en LRU?», que buscadas tal cual no se parecen a nada del corpus. Las técnicas que se aplican en este paso son:

Reescritura de la consulta [81]: un LLM reformula la pregunta para que se entienda por sí sola, resolviendo las referencias a la conversación anterior.

HyDE [82]: un LLM redacta una respuesta hipotética a la pregunta y se busca con ella, ya que una respuesta se parece más a los documentos que una pregunta. El inconveniente es que esa respuesta inventada puede llevar la búsqueda hacia contenido que no es de la asignatura.

Extracción de filtros: se identifican en la pregunta restricciones como un tema o un tipo de documento para acotar la búsqueda.

Se ha optado por la **reescritura de la consulta**, aplicada **solo cuando hay una conversación previa**. Una pregunta que se hace por primera vez ya se entiende sola, y reescribirla añadiría una llamada al LLM y un tiempo de espera sin ningún beneficio.

Para que esta llamada adicional no afecte al ritmo de la conversación, se utiliza un modelo rápido configurado para no razonar antes de contestar, y solo se le pasan los últimos turnos de la conversación recortados con el propósito de resolver un «eso» basta con saber de qué se estaba hablando.

Es importante destacar que la pregunta reescrita **solo se usa para buscar**: la respuesta se genera a partir de la pregunta original del usuario, de forma que el sistema responde exactamente a lo que se le ha preguntado.

La extracción de filtros se ha descartado: con un corpus de 33 documentos, la búsqueda ya encuentra el tema adecuado sin necesidad de acotarla, y un filtro mal extraído dejaría fuera la respuesta correcta.

5.3.2. Recuperación

En la sección 4.2.2 identificamos el principal reto de nuestra recuperación: el temario está lleno de términos exactos, como órdenes de *shell* o siglas como FIFO o LRU, pero los estudiantes preguntan con sus propias palabras. De las estrategias vistas en la sección 4.2.2:

Búsqueda semántica o densa [83]: compara los embeddings de la pregunta y de los *chunks*. Entiende las paráfrasis, pero puede pasar por alto un término técnico concreto.

Búsqueda por palabras clave: el algoritmo de referencia es **BM25** [84], una evolución de TF-IDF. Encuentra el término exacto, pero no entiende sinónimos ni paráfrasis.

Modelos intermedios, como SPLADE [85] o ColBERT [86], que intentan reunir las ventajas de ambas en un solo modelo, a cambio de un índice propio más complejo de construir.

Búsqueda híbrida: combina las dos primeras y fusiona sus resultados.

Hemos elegido la **búsqueda híbrida** precisamente porque cada una de las dos búsquedas cubre el punto débil de la otra. Cada una propone sus 20 mejores candidatos y ambas listas se fusionan con *Reciprocal Rank Fusion* (RRF) [87], un método sencillo que combina las posiciones de cada *chunk* en las dos listas, dando algo más de peso a la búsqueda semántica (60 %) que a la léxica (40 %).

Para que BM25 funcione bien en español hubo que filtrar las palabras vacías. Sin este filtro, en una pregunta como «¿qué es la paginación?» las palabras «qué», «es»

y «la» pesan tanto como «paginación», y la búsqueda devolvía documentos que solo coincidían en esas palabras. Esta lista hay que construirla con cuidado: la palabra «cuanto», que parece vacía, es en este temario el nombre del *quantum* de la planificación Round Robin, y filtrarla estropeaba todas las preguntas sobre ese algoritmo. Por el mismo motivo se planteó con cautela la reducción de las palabras a su raíz (*stemming*), ya que convierte en un mismo término palabras distintas del temario, como «proceso», «procesador» y «procesamiento». Se evaluó activándola y no mejoró los resultados (sección 6.3.2), por lo que se mantiene desactivada.

Reordenado (*re-ranking*)

Los embeddings comparan la pregunta y el *chunk* por separado, cada uno resumido en un vector. Un **modelo de reordenado** (*cross-encoder*) lee la pregunta y el *chunk* a la vez, por lo que distingue mucho mejor lo que realmente responde a la pregunta de lo que solo comparte palabras con ella [88]. Es más lento, por lo que solo se aplica sobre los candidatos que ya ha seleccionado la búsqueda híbrida. Las alternativas estudiadas son:

No reordenar: la opción más rápida, que se queda con el orden de la búsqueda híbrida.

mmarco-mMiniLMv2 [89]: un modelo pequeño [90] entrenado con mMARCO [91], un conjunto de datos de búsqueda traducido a varios idiomas, entre ellos el español. Se ejecuta en local en poco tiempo.

bge-reranker-v2-m3 [92]: un modelo multilingüe cinco veces más grande, capaz de leer textos mucho más largos.

Un LLM como reordenador [93]: se le pide a un LLM que ordene los candidatos. Da buenos resultados, pero añade una llamada más a la API en cada pregunta, con el coste en tiempo y en límite de peticiones que eso supone.

Hemos elegido **mmarco-mMiniLMv2**. Se evaluó también bge-reranker-v2-m3 y, aunque obtuvo resultados algo mejores, la mejora no fue estadísticamente significativa y, al no disponer de un equipo potente multiplicaba varias veces el tiempo de espera de cada respuesta.

La evaluación mostró además que **prescindir del reordenado** no empeora las respuestas y reduce el tiempo de espera a menos de la mitad (sección 6.3.2). Esto se debe a que mmarco-mMiniLMv2 es un modelo pequeño, trunca parte de los *chunks*

más largos y está entrenado sobre una traducción automática, mientras que la búsqueda densa ya utiliza un modelo de embeddings de gran tamaño. Aun así, se ha mantenido en la configuración del sistema para que todas las comparaciones partan de la misma configuración de referencia.

Orden, redundancia y número de *chunks*

En el capítulo 4 estudiamos que el sobrepaginado hace que los *chunks* contiguos de una misma sección se parezcan mucho entre sí, por lo que pueden ocupar varias posiciones del contexto repitiendo la misma idea. Una técnica clásica para evitarlo es MMR [94], que penaliza los resultados parecidos a los ya elegidos. En nuestro caso existe una solución más sencilla: como conocemos la sección de la que procede cada *chunk*, basta con **quedarse con un único *chunk* por sección**. Al hacerlo después del *reranking*, el que se conserva es siempre el mejor valorado.

Finalmente se pasan al LLM los **5 mejores *chunks***. Un número mayor no garantiza mejores respuestas: los LLM tienden a prestar menos atención a la información situada en la mitad de un contexto largo [95], y cada *chunk* adicional aumenta la cantidad de información irrelevante. Con cinco *chunks*, ordenados de más a menos relevante, se cubren también las preguntas que obligan a combinar varias secciones. Se evaluó también con tres y con ocho: más *chunks* aportan más información útil, pero también más información irrelevante, y la calidad de las respuestas no cambió (sección 6.3.2). Cinco queda como punto intermedio entre ambos efectos.

Un detalle de diseño es que la recuperación se realiza **una sola vez por pregunta**, y esos mismos *chunks* son los que recibe el LLM y los que se muestran al usuario como fuentes. Así se garantiza que las fuentes mostradas son exactamente las que ha leído el modelo.

5.3.3. Aumento del *prompt*

Como adelantamos en la sección 4.2.3, la plantilla del *prompt* es la principal herramienta para conseguir la trazabilidad que comentamos en 3.2.2. Los *chunks* se incluyen numerados y acompañados de su procedencia (documento y sección), para que el modelo pueda citarlos y el usuario pueda reconocer la fuente. Establecemos las siguientes reglas en la plantilla del *prompt*:

1. Responder únicamente con la información del contexto, aunque se permite com-

binar información de varios *chunks*.

2. Citar siempre el *chunk* del que sale cada afirmación con la marca [n].
3. Si la información es parcial, responder solo con lo que se pueda afirmar.
4. Si la información no está en el contexto, responder exactamente: «No tengo la suficiente información para contestar a su pregunta.»
5. No inventar, no citar fuentes externas y no utilizar conocimiento propio.
6. Responder de forma directa a lo que se pregunta, en pocas frases.

Las reglas 1, 4 y 5 previenen la inclusión de alucinaciones descritas en 3.1.7. Le prohíben al modelo completar con información plausible lo que no encuentra en la documentación. La regla 2 hace cada afirmación comprobable por el estudiante.

Se realizaron varias pruebas para determinar la regla 6. La primera versión del *prompt* pedía respuestas «claras, precisas y bien explicadas», y el modelo respondía con textos mucho más largos de lo necesario, llenos de detalles que no se habían preguntado. Se compararon tres estilos de respuesta (explicativo y conciso) y se adoptó el **conciso**, por resultar ser el más equilibrado entre los parámetros que mediremos en el siguiente capítulo. Esta comparación se detalla en la sección 6.3.1.

5.3.4. Generación

En la sección 4.2.4 establecimos cinco criterios para elegir el LLM: calidad y seguimiento de instrucciones, ventana de contexto, coste, latencia y reproducibilidad. Los candidatos estudiados a lo largo del proyecto han sido:

Llama 3.3 70B [96] a través de Groq: utilizado en la primera versión del prototipo por su latencia muy baja y su nivel gratuito. Se sustituyó al reunir todos los modelos del sistema (embeddings, generador y juez) en un único proveedor.

Gemini [97]: de muy buena calidad, pero como vimos en la sección 5.1.3, su crédito gratuito se agotó durante la evaluación.

Modelos de Blablador: entre los modelos disponibles de mayor tamaño se encuentran **GPT-OSS-120B** [98], un modelo abierto publicado por OpenAI. Los modelos de la familia **Qwen3** [99] y **MiniMax-M2.7**. Blablador ofrece también modelos pequeños, como **Apertus-8B** [100], un modelo código abierto (también denominado de pesos abiertos) de origen suizo de 8 000 millones de parámetros.

Se ha elegido **GPT-OSS-120B** por las siguientes razones:

- **Calidad y seguimiento de instrucciones:** redacta con corrección en español y respeta las reglas del *prompt*, citando las fuentes y rechazando las preguntas cuya respuesta no está en el contexto.
- **Respuesta limpia:** es un modelo que razona antes de contestar, pero entrega ese razonamiento por separado, de modo que el texto de la respuesta llega limpio. Con MiniMax-M2.7, en cambio, se mezclaba su razonamiento con la respuesta.
- **Coste y ventana de contexto:** es gratuito a través de Blablador y su ventana de contexto es de sobra suficiente para cinco *chunks*.
- **Independencia del juez:** como veremos en la evaluación, el juez debe ser de una familia distinta a la del generador. Elegir GPT-OSS como generador nos permite poder usar Qwen para que actúe como juez.

Para comprobar que el tamaño del generador importa, se evaluó también el sistema con Apertus-8B, manteniendo todo lo demás igual. Un modelo pequeño sigue peor las reglas del *prompt*: las respuestas empeoraron claramente y, lo que es más grave para un docente virtual, el modelo respondía a preguntas cuya respuesta no está en el temario en lugar de reconocer que no tenía la información (sección 6.3.3). No es, por tanto, una alternativa válida para nuestro caso de uso.

En cuanto a la reproducibilidad, el modelo se utiliza con **temperatura 0**, que es la configuración más estable posible. Aun así, un modelo servido a muchos usuarios a la vez no es completamente determinista. Es decir, aunque se utilice temperatura 0, una misma pregunta con los mismos *chunks* puede recibir respuestas ligeramente distintas. Este ruido se cuantifica y se tiene en cuenta al comparar configuraciones (sección 5.4.4).

Generador de pago como comparación

¿Mejoraría el sistema si se pagara por un generador más capaz? Para responderla se realizó un experimento en el que se sustituyó GPT-OSS-120B por **GPT-5.1** [47], manteniendo idéntico el resto de componentes del sistema: los mismos *chunks* para cada pregunta, el mismo *prompt* y el mismo juez. La elección de este modelo tiene un motivo: al ser del mismo fabricante que GPT-OSS-120B, la comparación enfrenta un modelo de pesos abiertos y gratuito con uno cerrado y de pago.

El diseño del experimento sigue las mismas reglas que el resto de comparaciones:

- **Solo cambia el generador.** El juez sigue siendo el mismo modelo gratuito de Blablador. Utilizar un juez de OpenAI habría cambiado a la vez lo que se mide y el instrumento de medida y, además, habría puntuado a dos modelos de su propia familia [101].
- **Identificador concreto:** el modelo se fija por su versión con fecha (gpt-5.1-2025-11-13) y no por su nombre genérico, por el mismo motivo que en Blablador.
- **Razonamiento reducido:** GPT-5.1 es, como GPT-OSS, un modelo que razona antes de contestar. Se configuró con el nivel de razonamiento bajo y sin parámetro de temperatura, ya que este tipo de modelos no admite fijarla.
- **Control del gasto:** cada llamada tiene un límite de *tokens* de salida, lo que acota el coste máximo de una evaluación completa por debajo del crédito disponible. Se registran los *tokens* y el coste de cada respuesta, y la evaluación se detiene sola antes de superar un presupuesto fijado. Una prueba previa con cinco preguntas permitió estimar el coste real antes de lanzar la evaluación completa. Esto se hizo para asegurar que el experimento se podía realizar con el crédito del que disponía.

Los resultados de este experimento se presentan en la sección 6.3.3.

5.3.5. Post-procesado

En la sección 4.2.5 se plantean dos comprobaciones sobre cada una de las respuestas antes de mostrarlas:

Comprobación de las citas: se extraen de la respuesta las marcas [n] y se descartan las que no corresponden a ningún *chunk* recibido, por ejemplo, un [9] cuando solo se entregaron cinco. Durante las pruebas se observó que el modelo, en ocasiones cita con un formato propio de su entrenamiento en lugar de [n]. Intentar corregirlo desde el *prompt* empeoró el problema, así que se optó por reconocer también ese formato. Si una respuesta no cita nada, se avisa al usuario.

Detección del rechazo: se detecta cuándo el sistema ha respondido con la frase de rechazo, para no mostrar fuentes en ese caso y para contabilizarlo en la evaluación. La frase es la siguiente: «No tengo suficiente información para responder a la pregunta».

Aunque se le pide al modelo que la escriba literalmente, a veces cambia una tilde o una mayúscula, por lo que la comparación se hace ignorando esas diferencias.

5.4. Evaluación y monitorización

Esta sección desarrolla la evaluación descrita en la sección 4.3. Hablaremos sobre el conjunto de evaluación, la herramienta que calcula las métricas, el modelo que actúa como juez y el método para comparar dos configuraciones del sistema.

5.4.1. Conjunto de evaluación

Siguiendo las características que establecimos en la sección 4.3.2, se ha construido un banco de **94 preguntas** sobre el temario, cada una con su respuesta de referencia (*ground truth*). Las respuestas de referencia se generaron con NotebookLM [102], una herramienta de Google que responde a partir de los documentos que se le proporcionan, a partir del mismo corpus, y después se revisaron a mano. Para que el banco sea exigente, las preguntas se reparten en tres niveles de complejidad:

- **Directas** (40): se responden con un único fragmento de los apuntes, como las preguntas de definición.
- **Síntesis** (39): obligan a combinar información de varias secciones o documentos.
- **Inferencia** (15): exigen razonar a partir de lo que dicen los documentos o tratan casos límite.

Todas estas preguntas tienen respuesta en el corpus, por lo que un rechazo cuenta siempre como un fallo. Para asegurarnos de que se está respondiendo con el contenido del corpus se ha preparado un segundo banco de 15 preguntas cuya respuesta no está en el corpus. En este banco lo correcto es responder usando la frase de rechazo expuesta anteriormente 5.3.5, y la métrica es el porcentaje de rechazos. Esta mide directamente si el sistema se inventa lo que no sabe.

5.4.2. Herramienta de evaluación

En la sección 4.3.3 vimos que necesitábamos una herramienta capaz de medir tanto la calidad de la recuperación como la de la respuesta. Las alternativas estudiadas son:

Métricas clásicas: BLEU [103] y ROUGE [104] cuentan las palabras que la respuesta comparte con la de referencia, y BERTScore [105] compara sus significados con embeddings. Son rápidas y deterministas, pero no contempla el hecho de que se puede responder de la misma manera usando otras palabras, es decir, una respuesta correcta redactada con otras palabras puntúa mal.

TruLens [106] y **DeepEval** [107]: herramientas completas de evaluación y monitorización, más orientadas a aplicaciones en producción.

RAGAS [108, 109]: herramienta de código abierto diseñada específicamente para evaluar sistemas RAG, con métricas para la recuperación y para la generación.

Hemos elegido **RAGAS** porque cubre los dos tipos de calidad que distinguimos en la sección 4.3.1, es gratuita, se integra con LangChain y es una de las herramientas más utilizadas en la literatura sobre evaluación de RAG. De sus métricas hemos seleccionado las siete de la tabla 5.3, agrupadas como en capítulo de diseño 4.

Tabla. 5.3: Métricas de evaluación utilizadas

Métrica	Qué mide
<i>Calidad de la recuperación</i>	
Precisión del contexto	Si los <i>chunks</i> recuperados son relevantes y están bien ordenados.
Cobertura del contexto	Si los <i>chunks</i> recuperados contienen todo lo necesario para responder.
<i>Calidad de la respuesta</i>	
Fidelidad	Si cada afirmación de la respuesta está respaldada por el contexto. Detecta alucinaciones.
Relevancia	Si la respuesta contesta a lo que se ha preguntado.
Corrección factual	Si las afirmaciones de la respuesta coinciden con las de la referencia, penalizando tanto lo que falta como lo que sobra.
Cobertura factual	Qué parte de la respuesta de referencia aparece en la respuesta.
Similitud semántica	Parecido de significado entre la respuesta y la referencia, medido con embeddings.

Para utilizar RAGAS en nuestro proyecto fueron necesarias dos adaptaciones:

- La primera tiene que ver con el idioma. RAGAS evalúa las respuestas mediante un modelo de lenguaje que actúa como juez, y ese juez sigue unas instrucciones internas que vienen escritas en inglés, mientras que nuestro corpus, las preguntas y las respuestas están en español. Para evitar que el idioma afecte a la calidad de la evaluación, traducimos esas instrucciones una sola vez y las guardamos. Así, todas las evaluaciones usan exactamente las mismas instrucciones y sus resultados son comparables entre sí, sin que una traducción distinta en cada ejecución pueda alterarlos.
- La segunda adaptación afecta a la métrica de corrección factual, que mide si lo que dice la respuesta coincide con la respuesta de referencia, y presentaba dos fallos. El primero era que no calculaba correctamente qué parte de la referencia queda cubierta por la respuesta. Por ejemplo, si la frase de referencia contiene dos datos y la respuesta solo recoge uno, la cobertura debería ser del 50 %. El segundo era que el juez leía las respuestas de referencia sin ver la pregunta, de modo que no entendía las muy breves que solo cobran sentido junto a ella. Una referencia como «En 1993» o «Sí, una vez» no significa nada por sí sola. Se corrigieron ambos problemas: se arregló el cálculo de la cobertura y se le proporciona la pregunta al juez junto con la referencia.

5.4.3. Modelo juez

Buena parte de las métricas de RAGAS necesitan un LLM que actúe como juez [110], por ejemplo para decidir si una afirmación está respaldada por el contexto. Para elegirlo hemos tenido en cuenta dos aspectos:

Debe ser de una familia distinta a la del generador: está demostrado que un LLM tiende a puntuar mejor los textos generados por sí mismo o por modelos de su misma familia [101].

Debe devolver su veredicto de forma limpia y en un tiempo razonable: RAGAS espera del juez una respuesta con un formato concreto y dentro de un tiempo límite. Por este motivo se descartó MiniMax-M2.7, que, como vimos, mezcla su razonamiento con la respuesta y RAGAS no es capaz de leerla.

Hemos elegido **Qwen3.8-Flash-Next** [99], de una familia distinta a GPT-OSS y disponible en Blablador. Por defecto este modelo razona antes de contestar, lo que

hacía que muchas evaluaciones superaran el tiempo límite de RAGAS y quedaran sin puntuar. Como lo que se le pide al juez (extraer las afirmaciones de un texto y comprobar si otro las respalda) no necesita ese razonamiento, se utiliza con el razonamiento desactivado. Este mismo modelo es el que se utiliza para la reescritura de consultas. Además, para no superar el límite de peticiones de Blablador, las llamadas al juez se realizan a un ritmo controlado.

El juez forma parte del instrumento de medida, y el instrumento de medida no puede cambiar entre las configuraciones que se comparan. Por ello, el juez, su configuración y los embeddings que usan las métricas son siempre los mismos en todas las evaluaciones, también en la del generador de pago. Para asegurarnos de que siempre se usa el mismo juez, cada evaluación deja constancia del juez y de las instrucciones traducidas con las que se puntuó, y el sistema se niega a continuar una evaluación a medias con un juez distinto.

5.4.4. Comparación entre configuraciones

Para saber si un cambio mejora el sistema, se evalúa dos veces cambiando **una sola cosa**, por ejemplo el modelo de reordenado o el estilo de respuesta, manteniendo idénticos el corpus, el índice, el generador y el juez. Así, cualquier diferencia en los resultados puede atribuirse al cambio realizado. Cada evaluación guarda la configuración con la que se realizó, y antes de comparar se comprueba automáticamente que dos configuraciones difieren únicamente en el parámetro estudiado.

Como vimos, ni el generador ni el juez son completamente deterministas, por lo que una diferencia pequeña entre dos evaluaciones puede deberse al azar. Para distinguir una mejora real del ruido se combinan tres herramientas:

- **El test de Wilcoxon pareado** [111]. Es pareado porque las dos configuraciones responden a las mismas preguntas, de modo que cada pregunta se compara consigo misma. Es no paramétrico, ya que no exige que las puntuaciones sigan una distribución normal, algo que no cumplen unas notas acotadas entre 0 y 1. Como se comparan siete métricas a la vez, se aplica además la corrección de Holm [112], que evita que alguna salga significativa solo por casualidad. Cuando se comparan más de dos configuraciones a la vez (por ejemplo, tres números de *chunks* o tres generadores) se utiliza su equivalente para varios grupos, el test de Friedman [113].
- Un **intervalo de confianza** para la diferencia media, obtenido por remuestreo

(*bootstrap*) [114] de las preguntas dentro de cada categoría, que indica no solo si hay diferencia sino de qué tamaño es.

- Una **banda de ruido**: la diferencia que cabe esperar al repetir la misma configuración sin cambiar nada. Se mide aprovechando que, en dos evaluaciones con el mismo generador, muchas preguntas reciben exactamente los mismos *chunks*. En esas preguntas, cualquier variación de la nota se debe únicamente al ruido del generador y del juez. Una diferencia que no supera esta banda no se considera una mejora, aunque el test la señale.

Por último, hay una medida que no depende de ningún LLM, la proporción de *chunks* que comparten dos configuraciones en cada pregunta. Indica directamente cuánto ha cambiado lo que lee el generador, sin el ruido del juez.

Para no ajustar el sistema a las propias preguntas con las que se evalúa, las decisiones de afinado (como el estilo de respuesta) se tomaron con un subconjunto de 24 preguntas de desarrollo, y después se comprobó que los resultados se mantenían en el resto del corpus. Los resultados de todas estas comparaciones se presentan en el capítulo 6.

5.5. Resumen de las tecnologías elegidas

La tabla 5.4 resume, para cada parte del sistema, la tecnología elegida y la principal alternativa descartada.

Tabla. 5.4: Resumen de las tecnologías elegidas para cada parte del sistema

Parte	Tecnología elegida	Principal alternativa
Framework	LangChain	LlamaIndex
Proveedor de modelos	Blablador	Gemini (de pago)
Conversión PDF a Markdown	Docling	Unstructured
<i>Chunking</i>	Por encabezados de Markdown	Tamaño fijo
Embeddings	Qwen3-Embedding-8B	BGE-M3
Base de datos vectorial	ChromaDB	FAISS
Preprocesado de consulta	Reescritura con historial	HyDE
Recuperación	Híbrida (BM25 + densa, RRF)	Solo densa
Reordenado	mmarco-mMiniLMv2	bge-reranker-v2-m3
Generación	GPT-OSS-120B	GPT-5.1, Apertus-8B
Evaluación	RAGAS	DeepEval
Juez	Qwen3.8-Flash-Next	MiniMax-M2.7
Interfaz	FastAPI + web propia	Streamlit

Capítulo 6

RESULTADOS

En este capítulo se presentan los resultados de la evaluación descrita en la sección 5.4. Primero se explica cómo se deben leer los resultados que obtenemos, después se analiza el rendimiento del sistema final y, por último, se presentan las comparaciones en las que se apoyan las decisiones de diseño del capítulo 5.

6.1. Cómo leer los resultados

6.1.1. Bancos de preguntas

Todas las evaluaciones utilizan los mismos dos bancos de preguntas:

Banco principal, de 94 preguntas. Tanto las preguntas como sus respuestas de referencia se crearon con NotebookLM [102] a partir del corpus de la asignatura, y después se revisaron cuidadosamente a mano. Todas tienen respuesta en el temario, por lo que, en este banco, que el sistema se niegue a responder cuenta siempre como un fallo. Las preguntas se reparten en tres tipos, cada uno con un propósito distinto, como recoge la tabla 6.1.

Banco de control, de 15 preguntas. Son preguntas cuya respuesta no se encuentra en el corpus. Su objetivo es asegurar que las respuestas del sistema se basan en el corpus y no en el conocimiento propio del modelo. En este banco la única respuesta correcta es la frase de control que impone el *prompt*:

«No tengo la suficiente información para contestar a su pregunta.»

Tabla. 6.1: Tipos de pregunta del banco principal.

Tipo	Propósito	Preguntas
Directas	Comprobar que el sistema encuentra y reproduce un dato o una definición que se encuentra en un único fragmento de los apuntes.	40
Síntesis	Comprobar que el sistema combina información repartida entre varias secciones o documentos.	39
Inferencia	Comprobar que el sistema razona a partir de lo que dicen los documentos, en casos límite cuya respuesta no aparece de forma literal.	15
Total		94

La métrica de este banco es el porcentaje de preguntas en las que el sistema responde con esa frase en lugar de inventarse una respuesta.

Todas las evaluaciones se puntúan con el mismo juez. Las siete métricas toman valores entre 0 y 1, donde 1 es el mejor resultado.

6.1.2. Métricas

Como métrica principal se utiliza la **corrección factual**, porque es la que mejor resume si una respuesta es correcta. Premia incluir lo que dice la respuesta de referencia y penaliza añadir afirmaciones que esta no contiene. Junto a ella, la **fidelidad** indica si la respuesta se ciñe a los documentos recuperados, es decir, si el sistema se inventa información.

La tabla 6.2 resume qué mide cada una de las siete métricas y cómo la obtiene el juez.

6.1.3. Configuración de referencia

Llamaremos **configuración de referencia** al sistema descrito en el capítulo anterior, que resume la tabla 6.3. Cada comparación de la sección 6.3 cambia un único elemento respecto a ella.

Tabla. 6.2: Métricas de la evaluación: qué mide cada una y cómo se obtiene.

Métrica	Qué mide	Cómo la obtiene el juez
<i>Calidad de la recuperación</i>		
Precisión del contexto	Si los <i>chunks</i> útiles están entre los recuperados y aparecen en las primeras posiciones.	Decide, <i>chunk</i> a <i>chunk</i> , si sirve para responder, y puntúa más alto cuando los útiles van primero.
Cobertura del contexto	Si los <i>chunks</i> recuperados contienen todo lo necesario para responder.	Divide la respuesta de referencia en afirmaciones y cuenta cuántas aparecen en los <i>chunks</i> .
<i>Calidad de la respuesta</i>		
Fidelidad	Si la respuesta se ciñe a los <i>chunks</i> recuperados. Detecta alucinaciones.	Divide la respuesta en afirmaciones y cuenta cuántas están respaldadas por los <i>chunks</i> .
Relevancia	Si la respuesta contesta a lo que se ha preguntado.	Escribe la pregunta a la que respondería esa respuesta y la compara con la pregunta real mediante embeddings.
Cobertura factual	Qué parte de la respuesta de referencia recoge la respuesta.	Cuenta cuántas afirmaciones de la referencia, leída junto a su pregunta, están respaldadas por la respuesta.
Corrección factual (principal)	Si la respuesta es correcta, penalizando tanto lo que le falta como lo que le sobra.	Combina, con una media armónica, la cobertura factual y la precisión factual: la parte de la respuesta respaldada por la referencia.
Similitud semántica	Parecido de significado entre la respuesta y la referencia.	No usa el juez: es la similitud del coseno entre los embeddings de ambos textos.

Tabla. 6.3: Configuración de referencia.

Elemento	Valor
Generador	GPT-OSS-120B, temperatura 0
Estilo de respuesta	Conciso
Embeddings	Qwen3-Embedding-8B
Recuperación	Híbrida: BM25 (40 %) y densa (60 %), 20 candidatos, fusión RRF
<i>Stemming</i>	Desactivado
Reordenado	mmarco-mMiniLMv2
<i>Chunks</i> al generador	5, uno por sección
Juez	Qwen3.8-Flash-Next, sin razonamiento

6.1.4. El ruido de la medida

Podemos realizar dos ejecuciones con la misma configuración y que salgan resultados diferentes, a esto le llamaremos ruido. Tenemos que definir cuánto varían los resultados cuando no se cambia nada. Para medirlo utilizaremos dos evaluaciones realizadas con el mismo generador en las que el modelo recibió exactamente los mismos *chunks*, en el mismo orden, en 68 preguntas. Podemos decir que cualquier diferencia en la nota se debe únicamente a que ni el generador ni el juez son deterministas, es decir, ante la misma entrada, no siempre producen la misma salida. La figura 6.1 muestra esas diferencias.

La idea general para obtener la banda de ruido es la misma que en cualquier medida experimental: toda medida tiene un error, y para conocer su tamaño se repite la medida **sin cambiar nada** y se observa cuánto varía. Lo que cambie entre las dos repeticiones no puede deberse al sistema, sino solo al error de la medida. El procedimiento tiene tres pasos:

1. **Diferencia por pregunta.** Para cada pregunta se resta la nota de la segunda repetición menos la de la primera. Si la medida fuera perfecta, todas las diferencias serían cero.
2. **Dispersión de las diferencias.** Se resume cuánto se alejan esas diferencias de cero con su **desviación típica**. Si las diferencias se reparten de forma aproximadamente normal, el 95 % de ellas cae dentro de $\pm 1,96$ desviaciones típicas: ese es el margen de error de una pregunta suelta.
3. **Paso a la media.** Al comparar configuraciones no se comparan preguntas sueltas, sino la media de muchas. Al promediar, unas notas suben y otras bajan, y los errores se compensan en parte: la desviación típica de la media de n preguntas es la de una pregunta dividida por $\sqrt{n}$.

Juntando los tres pasos, la banda de ruido de una métrica es:

$$\text{banda} = \pm 1,96 \cdot \frac{\text{desviación típica de las diferencias}}{\sqrt{n}}$$

es decir, el margen dentro del cual cae, el 95 % de las veces, la diferencia entre las medias de dos evaluaciones de n preguntas idénticas.

Como podemos observar en la figura 6.1 solo 21 de las 68 preguntas repitieron nota, las otras 47 variaron sin que se cambiara nada. De hecho, solo 3 de las 68

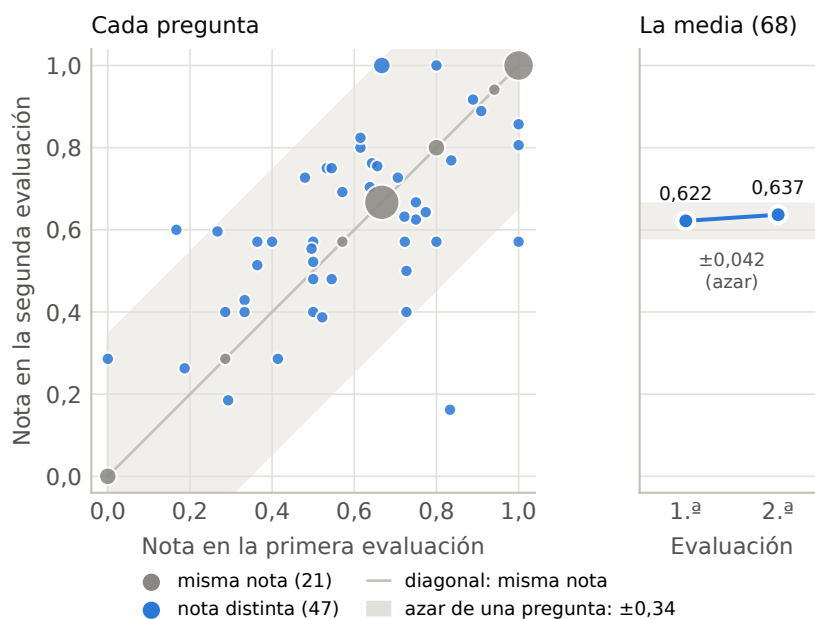


Figura 6.1: Cambio de la corrección factual de cada pregunta entre dos evaluaciones en las que el modelo recibió exactamente los mismos *chunks*. En gris, las preguntas que repitieron nota, en azul, las que cambiaron. La parte sombreada la llamaremos banda de ruido, la cual alberga las notas que consideramos que han variado por obra del azar.

respuestas coincidieron palabra por palabra. Los cambios son a menudo grandes: en 15 preguntas, más de una de cada cinco, la nota varió 0,2 o más, y en una llegó a caer 0,67. En consecuencia, la nota de una pregunta concreta es poco fiable. La desviación típica de estas 68 diferencias es de 0,176, así que, aplicando el segundo paso, por puro azar una pregunta puede variar en torno a $\pm 1,96 \cdot 0,176 \approx \pm 0,34$. Ese margen, marcado por la línea superior de la figura, contiene 65 de los 68 cambios observados.

Al realizar el promedio de muchas preguntas estas variaciones se compensan en gran parte, porque unas notas suben y otras bajan. Aplicando el tercer paso, la media de 94 preguntas varía unas diez veces menos que una pregunta aislada (la raíz cuadrada de 94 es 9,7), es decir, $\pm 1,96 \cdot 0,176 / \sqrt{94} \approx \pm 0,036$. Ese margen es la **banda de ruido**: la diferencia en la media que cabe esperar entre dos evaluaciones de 94 preguntas sin cambiar nada. Calculada del mismo modo para las demás métricas, es de $\pm 0,032$ para la cobertura factual y la cobertura del contexto, $\pm 0,023$ para la fidelidad y $\pm 0,011$ para la precisión del contexto.

A lo largo del capítulo se aplica el siguiente criterio: **una diferencia entre dos configuraciones solo se considera real si supera la banda de ruido y, además, es estadísticamente significativa**, es decir, si el test de Wilcoxon, tras la corrección de Holm, da un valor p inferior a 0,05. El valor p indica lo probable que sería encon-

trar una diferencia al menos tan grande si las dos configuraciones fueran en realidad equivalentes; cuanto más pequeño es, menos probable es que la diferencia se deba al azar. Las diferencias que no cumplen ambas condiciones se tratan como empates. En las figuras de comparación, la banda de ruido se dibuja siempre como una franja gris: lo que cae dentro de ella no se distingue del azar.

El valor p y el umbral de 0,05. Exigir un valor p inferior a 0,05 equivale a aceptar, como mucho, un 5 % de riesgo de dar por buena una diferencia que en realidad no existe.

El test de Wilcoxon pareado [111] es el que proporciona ese valor p al comparar dos configuraciones. Funciona en cuatro pasos:

1. Para cada pregunta se calcula la diferencia de nota entre las dos configuraciones.
2. Se ordenan esas diferencias por su tamaño, sin tener en cuenta el signo, y se numeran de la más pequeña a la más grande. Ese número es el *rango* de cada diferencia.
3. Se suman por separado los rangos de las preguntas en las que gana cada configuración.
4. Si las dos configuraciones fueran equivalentes, cada una ganaría en unas preguntas y perdería en otras por cantidades parecidas, y las dos sumas serían similares. El valor p mide lo improbable que es, en ese caso, un desequilibrio tan grande como el observado.

Es un test *pareado* porque las dos configuraciones responden a las mismas preguntas, de modo que cada pregunta se compara consigo misma: así no influye que unas preguntas sean más difíciles que otras. Además, al trabajar con rangos no exige que las notas sigan una distribución normal, algo que no cumplen unas notas acotadas entre 0 y 1. Cuando se comparan más de dos configuraciones a la vez, como los tres números de *chunks* de la sección 6.3.2, se utiliza su equivalente para varios grupos, el test de Friedman [113].

La corrección de Holm [112]. En cada comparación se examinan siete métricas, es decir, se hacen siete tests a la vez. Aunque las dos configuraciones fueran equivalentes, la probabilidad de que al menos uno de los siete diera un valor p inferior a 0,05 por pura casualidad sería de cerca del 30 % ($1 - 0,95^7$). La corrección de Holm lo evita: ordena los siete valores p de menor a mayor y multiplica el menor por 7, el siguiente por 6, y así sucesivamente, antes de compararlos con 0,05. Por ejemplo, al comparar

bge-reranker-v2-m3 con la configuración de referencia (sección 6.3.2), la corrección factual da un valor p de 0,034, que por sí solo sería significativo; multiplicado por 7 pasa a 0,24, y deja de serlo.

6.2. Rendimiento del sistema final

La tabla 6.4 recoge los resultados de la configuración de referencia sobre las 94 preguntas, en total y por tipo de pregunta.

Tabla. 6.4: Resultados del sistema final sobre el banco de 94 preguntas. Valores entre 0 y 1, cuanto más alto mejor.

Métrica	Total	Directas	Síntesis	Inferencia
<i>Calidad de la respuesta</i>				
Corrección factual	0,621	0,631	0,613	0,616
Cobertura factual	0,792	0,867	0,764	0,666
Fidelidad	0,915	0,967	0,916	0,763
Relevancia	0,793	0,809	0,795	0,745
Similitud semántica	0,807	0,777	0,865	0,740
<i>Calidad de la recuperación</i>				
Precisión del contexto	0,787	0,840	0,763	0,710
Cobertura del contexto	0,870	0,962	0,870	0,622
Preguntas	94	40	39	15
Rechazos (fallos)	2	0	1	1

De estos resultados se extraen cuatro observaciones:

El sistema no se inventa la información. La fidelidad media es de 0,915 y llega a 0,967 en las preguntas directas, es decir, casi todo lo que afirman las respuestas está respaldado por los *chunks* recuperados. Además, en el banco de preguntas fuera del temario el sistema rechazó las 15, y en el banco principal solo se negó a responder en 2 de las 94 preguntas.

Las respuestas son correctas, aunque algo más amplias que la referencia. La cobertura factual (0,792) indica que las respuestas recogen, de media, unas cuatro quintas partes de lo que dice la respuesta de referencia. La corrección factual es más baja (0,621) porque también penaliza las afirmaciones que no están en la referencia, y las respuestas del sistema son unas dos veces más largas que esta, una mediana de 76 palabras frente a 36. Como la fidelidad es alta podemos decir que esos detalles adicionales proceden de los documentos, es decir, son información correcta que las respuestas de referencia por ser más breves no incluyen.

Todas las respuestas citan sus fuentes. Las 92 respuestas que el sistema responde incluyen al menos una cita. En 25 de ellas el modelo utilizó su propio formato de cita en lugar de [n], que el post-procesado reconoce igualmente.

El tiempo de respuesta es aceptable y no tiene coste de uso. La mediana del tiempo de respuesta es de 5,9 segundos por pregunta, y el 90 % de las respuestas tarda menos de 8,1 segundos. El coste por pregunta es nulo, ya que todos los modelos se utilizan de forma gratuita.

6.2.1. Dónde falla el sistema

Por tipo de pregunta, las más difíciles son las de inferencia. Su corrección factual es similar a la del resto, pero la cobertura del contexto cae a 0,622, es decir, los *chunks* recuperados contienen, de media, el 62 % de lo necesario para responder frente al 96 % en las preguntas directas. Es también donde más baja la fidelidad (0,763), porque cuando falta información el modelo tiende a completarla.

Para comprobar si el problema está en la recuperación o en la generación, la tabla 6.5 separa las preguntas según si los *chunks* recuperados contenían todo lo necesario para responder (cobertura del contexto igual a 1) o solo una parte.

Tabla. 6.5: Resultados del sistema final según si el contexto recuperado estaba completo.

Contexto	Preguntas	Corrección factual	Cobertura factual	Fidelidad
Completo	71	0,682	0,864	0,962
Parcial	23	0,434	0,570	0,763

Cuando la recuperación encuentra toda la información, el generador responde bien y casi sin salirse de los documentos. Cuando solo encuentra una parte, todas las métricas caen. Las medias, sin embargo, no dicen cómo se reparten las notas. Una corrección factual media de 0,62 podría obtenerse de dos formas muy distintas: que casi todas las respuestas sean correctas a medias, con notas en torno a 0,6, o que haya muchas respuestas casi perfectas junto a unas pocas muy malas. Las dos situaciones apuntan a problemas diferentes. En la primera, el sistema dejaría fuera parte de la respuesta en casi todas las preguntas, lo que indicaría un problema general del generador. En la segunda, el sistema funcionaría bien en la mayoría de los casos y fallaría en preguntas concretas, cuya causa puede buscarse una a una. Para distinguirlas, la figura 6.2 muestra la nota de cada pregunta por separado.

Si nos fijamos en la figura 6.2 el eje horizontal (corrección factual) va de 0 (nada de

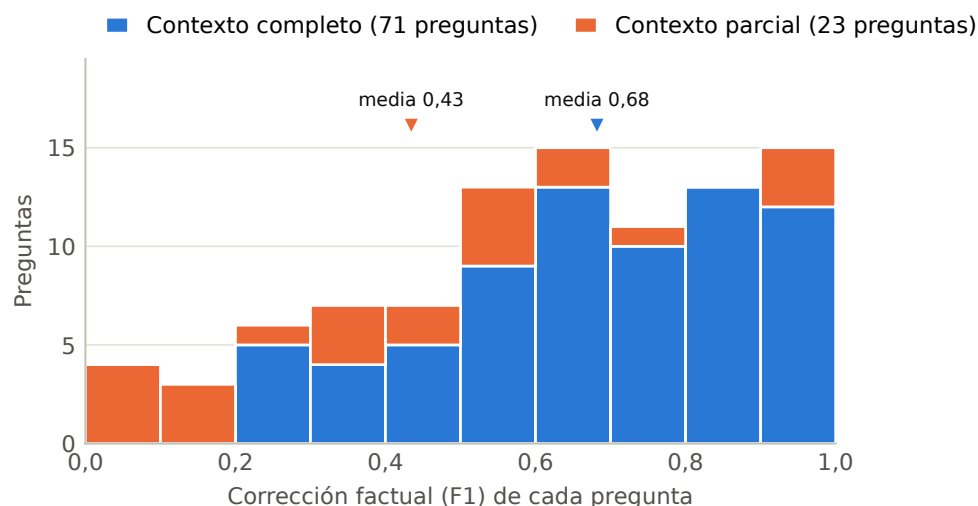


Figura 6.2: Corrección factual de cada una de las 94 preguntas del sistema final, según si los *chunks* recuperados contenían todo lo necesario para responder (contexto completo) o solo una parte (contexto parcial). Cada barra cuenta las preguntas con una nota en ese tramo. Los triángulos marcan la media de cada grupo.

lo que dice la respuesta coincide con la referencia) a 1 (coincide por completo), y en el eje vertical podemos ver el número de preguntas con una nota en ese tramo. Las barras se dividen por colores. En azul tenemos las preguntas con el contexto completo, y en naranja, las de contexto parcial. Por ejemplo, la barra de la derecha indica que 15 preguntas obtuvieron una nota entre 0,9 y 1: 12 con el contexto completo y 3 con el parcial.

Podemos observar que la mayor parte de las preguntas se concentra en la mitad derecha: 67 de las 94 obtienen 0,5 o más, y 13 alcanzan la nota máxima. Obtenemos pocos fallos graves pero se deben al mismo factor, las 7 preguntas con peor nota, por debajo de 0,2, son todas de contexto parcial, y por eso las dos primeras barras son naranjas en su totalidad. En el extremo contrario, 25 de las 28 preguntas con una nota de 0,8 o más tenían el contexto completo. Dicho de otro modo, con el contexto completo solo 5 de las 71 preguntas (un 7 %) bajan de 0,3, mientras que con el contexto parcial lo hacen 8 de 23 (un 35 %).

Los dos rechazos del banco pertenecen también al grupo de contexto parcial. En la pregunta 85, por ejemplo, ningún *chunk* recuperado trataba el riesgo de seguridad por el que se preguntaba. **El principal límite del sistema es, por tanto, la recuperación, y no la generación.** La sección 6.3.3 lo confirma desde otro ángulo.

6.3. Comparación de alternativas

Esta sección presenta las comparaciones que justifican las decisiones de diseño. En todas ellas se cambia un solo elemento de la configuración de referencia y se mantienen idénticos los demás, incluido el juez. Al final de la sección, la figura ?? reúne todas las comparaciones en un único gráfico.

6.3.1. Estilo de respuesta

Los dos estilos de respuesta comparados solo se diferencian en dos piezas del *prompt*: el objetivo que se le da al modelo y la regla de estilo. El resto del *prompt* es idéntico, de modo que la comparación mide únicamente el efecto de esas dos piezas. La tabla 6.6 recoge sus textos.

Tabla. 6.6: Diferencias entre los *prompts* de los dos estilos de respuesta.

Estilo	Objetivo: respuestas...	Regla de estilo
Explicativo (original)	«claras, precisas y bien explicadas»	«Sé claro y ordenado. Usa listas cuando ayuden.»
Conciso (elegido)	«claras, precisas y concisas»	«Responde de forma directa a lo que se pregunta, en pocas frases. No añadas contexto, ejemplos ni detalles que la pregunta no pida.»

La tabla 6.7 muestra los dos estilos sobre el banco completo.

Tabla. 6.7: Comparación de los dos estilos de respuesta sobre las 94 preguntas. La longitud es la mediana del número de palabras de cada respuesta; como comparación, se incluye la de las respuestas de referencia.

Estilo	Palabras	Corrección factual	Cobertura factual	Fidelidad
Explicativo	171	0,564	0,855	0,875
Conciso (elegido)	76	0,621	0,792	0,915
Referencia	36	—	—	—

El estilo explicativo genera respuestas más de dos veces más largas que el conciso y casi cinco veces más largas que la referencia. Cubre algo más de la referencia, pero añade tantas afirmaciones que no se piden que su corrección factual cae 0,057, por encima de la banda de ruido, aunque sin llegar a ser significativa tras la corrección de Holm ($p = 0,07$). Se mantuvo el **conciso** por ser el que mejor equilibra responder a lo que se pregunta sin añadir lo que no. Además, su corrección factual en las 24

preguntas de desarrollo (0,629) es prácticamente igual a la del resto del banco (0,619), lo que indica que la elección no está ajustada a las preguntas con las que se tomó.

6.3.2. Recuperación

Reordenado

La figura 6.3 compara las tres opciones de *reranking* evaluadas: sin *reranking*, *reranking* con mmarco-mMiniLMv2 (la opción elegida) y *reranking* con bge-reranker-v2-m3.

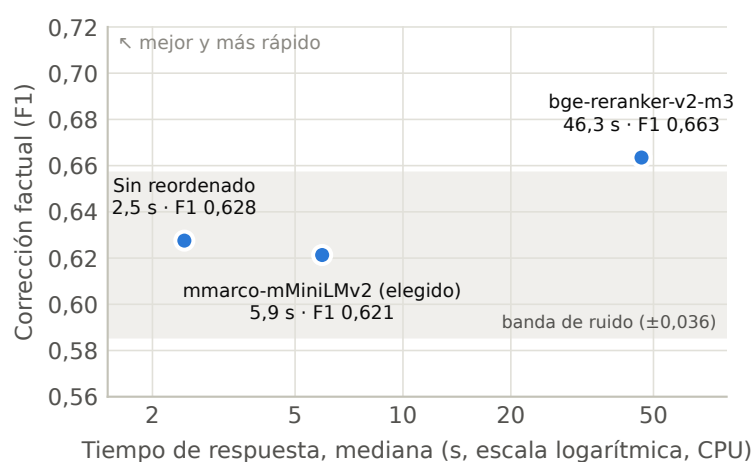


Figura 6.3: Corrección factual frente al tiempo de respuesta, sin reordenado y con los dos modelos de reordenado evaluados. Cada punto es una evaluación completa de 94 preguntas, su altura es la corrección factual media, y su posición horizontal, la mediana del tiempo de respuesta. La franja gris es la banda de ruido alrededor de la opción elegida.

Cómo se lee. Cada punto resume una evaluación completa del sistema con una de las tres opciones. Su altura es la calidad de las respuestas (la corrección factual media) y su posición horizontal, lo que tarda el sistema en responder (la mediana del tiempo por pregunta). Lo deseable es estar arriba y a la izquierda: mejor y más rápido. El eje horizontal usa una escala logarítmica, en la que cada marca multiplica el tiempo en lugar de sumarlo; así caben en la misma figura 2,5 y 46 segundos. La franja gris es la banda de ruido centrada en la opción elegida: un punto dentro de ella no se distingue de volver a evaluar esa misma opción.

Qué se observa. Las tres opciones ofrecen una calidad parecida, y la diferencia real está en el tiempo de respuesta:

- **Sin reordenado**, el punto queda dentro de la franja gris: su corrección factual

es prácticamente la misma que con mmarco-mMiniLMv2 (0,628 frente a 0,621, un empate), y la respuesta llega en menos de la mitad de tiempo (2,5 frente a 5,9 segundos). Solo el 40 % de los *chunks* coincide entre ambas opciones: el reordenado cambia mucho lo que lee el modelo, pero no mejora el resultado.

- **bge-reranker-v2-m3** es el único punto por encima de la franja: obtiene la mejor corrección factual (0,663, es decir, 0,042 más). Es la única mejora de todas las comparaciones que supera la banda de ruido, pero no es significativa tras la corrección de Holm ($p = 0,24$) y multiplica por ocho el tiempo de respuesta (46,3 segundos).

Con estos datos, el reordenado actual no aporta una mejora medible. Cabe preguntarse, entonces, por qué todas las evaluaciones se realizaron con él. Los motivos son los siguientes:

- **Se eligió antes de medir.** La configuración de referencia se fijó al inicio de la evaluación siguiendo la práctica recomendada en la literatura, que es reordenar con un *cross-encoder* los candidatos de la búsqueda [88, 36]. mmarco-mMiniLMv2 era la opción natural: multilingüe, entrenado también en español y capaz de funcionar sin tarjeta gráfica.
- **Hacía falta una referencia fija.** Para que las comparaciones sean válidas, todas deben medirse frente a la misma configuración. El estilo de respuesta, el número de *chunks*, el *stemming* y los generadores se compararon frente a la configuración con mmarco-mMiniLMv2.
- **No reordenar no es mejor, sino un empate.** Que el reordenado no aportaba nada solo se supo al evaluar la alternativa sin él, y la diferencia (+0,006 de corrección factual) queda dentro de la banda de ruido ($\pm 0,036$), con un valor p de 1,0. Es decir, los datos no permiten afirmar que una opción sea mejor que la otra.
- **Cambiar la referencia tenía un coste alto y ningún beneficio demostrado.** Sustituirla habría obligado a repetir todas las evaluaciones, de unas tres horas de puntuación cada una por el límite de peticiones del juez, para seguir comparando lo mismo.
- **Deja el sistema preparado para mejorar.** Mantener la etapa de reordenado permite pasar a bge-reranker-v2-m3, la única alternativa que supera la banda de ruido, cambiando solo el modelo, si se dispone de una tarjeta gráfica.

En un despliegue sin tarjeta gráfica, prescindir del reordenado sería una simplificación razonable: ofrece la misma calidad en menos de la mitad de tiempo.

Número de *chunks*

La figura 6.4 muestra qué ocurre al variar el número de *chunks* que recibe el generador.

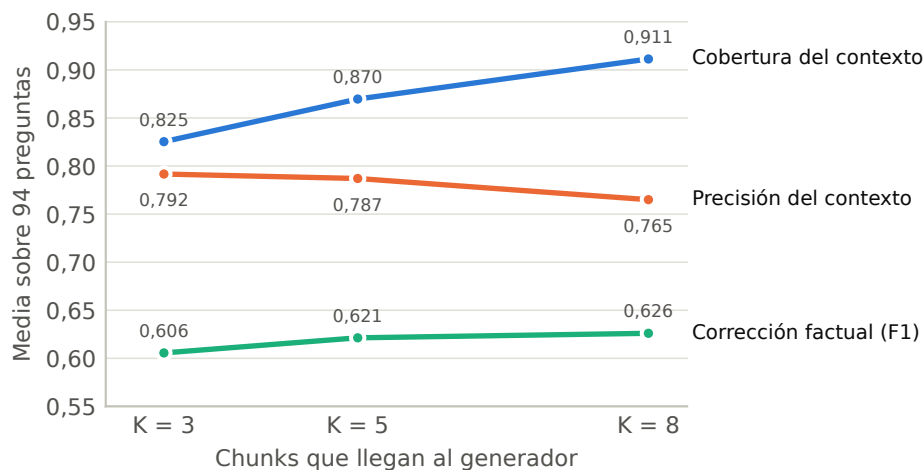


Figura 6.4: Efecto del número de *chunks* que llegan al generador sobre dos métricas de la recuperación (cobertura y precisión del contexto) y sobre la calidad de la respuesta (corrección factual). Media sobre las 94 preguntas.

Como podemos ver en la figura 6.4 las dos líneas relacionadas con la recuperación (*Cobertura del contexto* y *Precisión del contexto*) se mueven en sentidos opuestos. Con más *chunks* el LLM obtiene más información de la necesaria (la cobertura del contexto sube de 0,825 a 0,911), pero también más información irrelevante (la precisión baja de 0,792 a 0,765). De estos cambios, solo el de la precisión es significativo (test de Friedman, $p = 0,021$ tras la corrección de Holm). La línea de la respuesta, en cambio, es casi plana (0,606, 0,621 y 0,626): ni pasar de 3 a 5 *chunks* ni pasar de 5 a 8 cambia la corrección factual más allá de la banda de ruido. Para hacer que el proceso sea más rápido se ha optado por recuperar cinco *chunks* el cual nos ofrece un equilibrio razonable.

Stemming

Activar el *stemming* en la búsqueda léxica no tuvo efecto: en 68 de las 94 preguntas el contexto recuperado fue exactamente el mismo, y la corrección factual varió 0,017, dentro de la banda de ruido. Hay cuatro razones que explican este resultado:

- **Solo afecta a una parte de la búsqueda.** El *stemming* solo modifica la búsqueda léxica, que pesa un 40 % en la fusión. La búsqueda semántica, que pesa el 60 % restante, no cambia.

- **La búsqueda léxica solo propone candidatos.** De los 20 candidatos de la búsqueda híbrida, el reordenado elige después los 5 que llegan al generador. Un cambio pequeño en el orden que propone BM25 rara vez altera esa selección final.
- **Sus efectos se compensan.** El *stemming* permite que coincidan formas distintas de una misma palabra, como «proceso» y «procesos», pero también funde términos diferentes del temario: «proceso», «procesador» y «procesamiento» quedan reducidos a la misma raíz, igual que «página» y «paginación». Lo que gana en unas preguntas lo pierde en otras.
- **Las preguntas usan el vocabulario de los apuntes.** Al estar formuladas con los mismos términos técnicos que los documentos, la coincidencia exacta de palabras ya funciona sin necesidad de reducirlas a su raíz.

Se mantiene desactivado.

6.3.3. Generador

Se compararon tres generadores con los mismos *chunks*, el mismo *prompt* y el mismo juez: Se comparan GPT-OSS-120B, Apertus-8B y un modelo comercial de pago, GPT-5.1. La tabla 6.8 resume los resultados.

Tabla. 6.8: Comparación de los tres generadores evaluados, con el mismo contexto, el mismo *prompt* y el mismo juez.

	Apertus-8B	GPT-OSS-120B	GPT-5.1
Acceso	Gratuito	Gratuito	De pago
Corrección factual	0,491	0,621	0,590
Cobertura factual	0,666	0,792	0,750
Fidelidad	0,780	0,915	0,896
Rechazos en el banco principal (fallos)	5	2	5
Rechazos fuera del temario (aciertos)	3 de 15	15 de 15	15 de 15
Respuestas sin ninguna cita	9	0	0
Tiempo de respuesta (mediana)	8,2 s	5,9 s	5,9 s
Coste por pregunta	0	0	0,005 \$

Apertus-8B es claramente peor. Su corrección factual es 0,128 inferior a la de GPT-OSS-120B y su fidelidad, 0,132 inferior, ambas diferencias significativas ($p = 0,002$ y $p = 0,001$). Lo más grave es su comportamiento fuera del temario, y es que respondió 12 de las 15 preguntas de control cuya respuesta no está en los documentos, en lugar de reconocer que no tenía la información suficiente además de no citar

ninguna fuente en 9 de sus respuestas. Un modelo pequeño no respeta de forma fiable las reglas del *prompt*.

GPT-5.1, el modelo de pago, no mejora al gratuito. Obtiene resultados ligeramente inferiores a los de GPT-OSS-120B en todas las métricas, aunque ninguna diferencia es significativa. Cuando ambos modelos responden (89 preguntas), sus resultados son prácticamente iguales. Corrección factual de 0,638 y 0,623 ($p = 0,66$). La diferencia en el total se debe a que GPT-5.1 se negó a responder a cinco preguntas, frente a dos de GPT-OSS-120B, y dos de sus tres rechazos corresponden a preguntas con el contexto incompleto.

La figura 6.5 explica este comportamiento separando de nuevo las preguntas según el contexto recuperado.

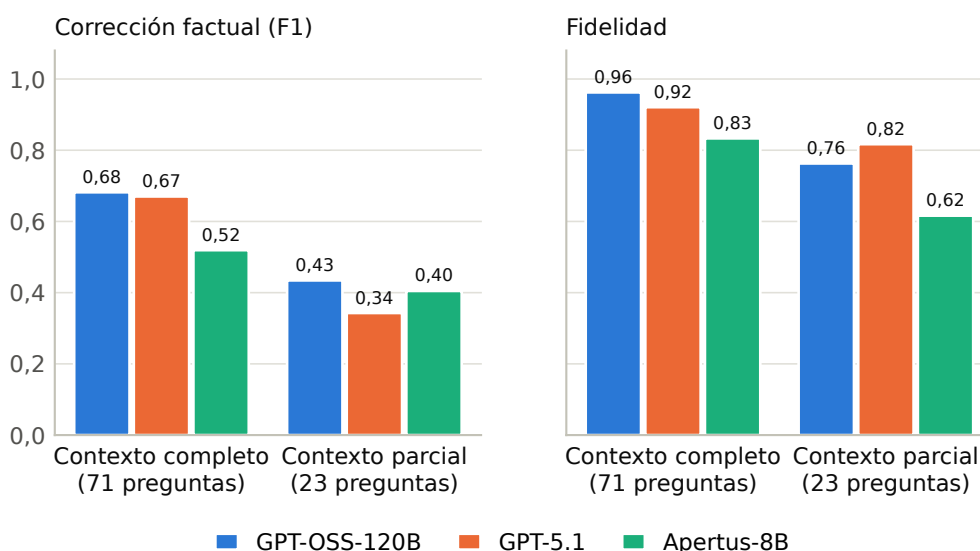


Figura 6.5: Corrección factual (izquierda) y fidelidad (derecha) de los tres generadores, separando las preguntas según si los *chunks* recuperados contienen todo lo necesario para responder (contexto completo) o solo una parte (contexto parcial). Los tres generadores recibieron exactamente los mismos *chunks* en cada pregunta.

La figura 6.5 tiene dos paneles, uno por métrica. En cada uno, las barras se agrupan según el contexto que recibió el generador. Completo (71 preguntas) o parcial (23). Dentro de cada grupo hay una barra por generador. Como los tres recibieron los mismos *chunks*, las diferencias entre las barras de un mismo grupo se deben solo al modelo, y las diferencias entre los dos grupos, a lo que encontró la recuperación.

Podemos observar que con el contexto completo, GPT-OSS-120B y GPT-5.1 empatan en corrección factual (0,68 y 0,67) y ambos se ciñen a los documentos (fidelidad de 0,96 y 0,92). Con el contexto parcial, GPT-5.1 se muestra más prudente, es decir, prefiere no responder o responder solo lo que puede respaldar, por lo que su correc-

ción factual es la más baja (0,34 frente a 0,43) y su fidelidad, la más alta (0,82 frente a 0,76). Apertus-8B queda por debajo en casi todo. Su única excepción es la corrección factual con contexto parcial (0,40), donde supera a GPT-5.1 porque responde siempre, aunque para ello añade información que no está en los *chunks*, como refleja su fidelidad de 0,62. Por esa misma prudencia, GPT-5.1 es algo mejor que GPT-OSS-120B en las preguntas directas (0,669 frente a 0,631) y peor en las de síntesis e inferencia, que son las que dependen de combinar información que no siempre se recupera completa.

Consideramos que lo más importante de la figura es el salto entre los dos grupos: en los tres generadores, la corrección factual con contexto parcial es mucho menor que con el completo. Ningún generador compensa la información que la recuperación no ha encontrado, lo que confirma lo visto en la sección [6.2.1](#).

En cuanto al coste, GPT-5.1 costó 0,51 dólares por las 109 preguntas de los dos bancos, unos 0,005 dólares por pregunta.

El experimento completo, incluida la prueba inicial y una repetición, costó 0,87 dólares. La única ventaja clara de GPT-5.1 es de formato, es decir, nunca utilizó el formato de cita propio de GPT-OSS-120B. Tampoco es más estable, al repetir la evaluación, solo 6 de sus 94 respuestas coincidieron palabra por palabra con las de la primera, frente a 5 de GPT-OSS-120B.

Capítulo 7

CONCLUSIONES

En este trabajo se ha desarrollado un docente virtual para la asignatura de Sistemas Operativos, un sistema RAG que responde a las preguntas de los estudiantes utilizando únicamente la documentación oficial de la asignatura y que indica de qué fragmento procede cada afirmación. Este capítulo valora el cumplimiento de los objetivos planteados en la introducción, resume las principales conclusiones del trabajo, señala sus limitaciones y propone líneas de trabajo futuro.

7.1. Cumplimiento de los objetivos

A continuación se revisa cada uno de los objetivos específicos definidos en la introducción.

Estudio teórico. Cumplido. Se ha estudiado el estado del arte actual en la arquitectura RAG, sus componentes, los modelos de embeddings y los LLM, y se han comparado las alternativas disponibles para cada parte del sistema (capítulo 5).

Definición de requisitos. Cumplido. Se eligió la asignatura de Sistemas Operativos, con un corpus de 33 documentos entre apuntes, guiones de prácticas y relaciones de problemas, y se fijaron dos requisitos que han guiado todas las decisiones: que el sistema fuera gratuito y poder obtener la mayor trazabilidad posible hasta las fuentes de cada una de las respuestas cada respuesta indicara sus fuentes.

Implementación del prototipo. Cumplido. El sistema funciona la conversión de los PDF hasta la respuesta citada, y cuenta con una interfaz web que permite conversar con el docente virtual, consultar los documentos y observar cómo funciona la

recuperación por dentro.

Diseño de pruebas. Cumplido. Se ha construido un banco de 94 preguntas en tres niveles de complejidad, desde preguntas directas hasta preguntas de inferencia, y un segundo banco de 15 preguntas cuya respuesta no está en el temario. Se evalúan con siete métricas y con un método de comparación que tiene en cuenta el ruido de la propia medida.

7.2. Conclusiones principales

1. **Es posible construir un docente virtual fiable sin coste.** El sistema final se ciñe a los documentos en la gran mayoría de sus afirmaciones (fidelidad de 0,915), cita sus fuentes en todas las respuestas, rechazó las 15 preguntas cuya respuesta no está en el temario y responde en unos 6 segundos. Todo ello sin coste por consulta, gracias a los modelos abiertos que ofrece gratuitamente un servicio académico.
2. **La calidad del sistema está limitada por la recuperación.** Cuando los *chunks* recuperados contienen toda la información necesaria, las respuestas son buenas; cuando solo contienen una parte, empeoran todas las métricas. Sustituir el generador por un modelo comercial más potente no mejoró los resultados, mientras que la única mejora que superó el ruido de la medida vino de la recuperación, con un modelo de reordenado más grande.
3. **El tamaño del generador importa, pero solo hasta cierto punto.** Un modelo pequeño no es una alternativa válida: se inventa respuestas a preguntas que no están en el temario. En cambio, a partir de un modelo abierto de gran tamaño, pagar por uno comercial no aporta una mejora medible.
4. **Medir bien es tan importante como construir.** La misma configuración, evaluada dos veces, no da las mismas respuestas ni las mismas notas. Sin cuantificar ese ruido, varias diferencias pequeñas se habrían tomado por mejoras. También fue necesario adaptar la herramienta de evaluación a nuestro caso, y la propia métrica de corrección factual, que penaliza responder de más, fue la que llevó a adoptar un estilo de respuesta más conciso.
5. **Añadir componentes no garantiza un sistema mejor.** El reordenado con un modelo pequeño y el *stemming* de la búsqueda léxica no aportaron ninguna mejora medible. Evaluar cada pieza por separado permite distinguir lo que realmente mejora el sistema de lo que solo lo hace más complejo o más lento.

7.3. Limitaciones

Los resultados de este trabajo deben interpretarse teniendo en cuenta las siguientes limitaciones:

- **Evaluación automática.** Las métricas las calcula un LLM que actúa como juez, y su criterio no se ha contrastado con valoraciones de profesores o estudiantes. Además, las respuestas de referencia se generaron con NotebookLM y, aunque se revisaron a mano, pueden reflejar su forma de responder.
- **Tamaño del banco de preguntas.** Con 94 preguntas solo se pueden detectar diferencias de cierto tamaño, en torno a 0,04 en la corrección factual. Las preguntas de inferencia, que son las más difíciles, son solo 15.
- **Resultados no deterministas.** Repetir una evaluación da resultados ligeramente distintos. Este ruido se ha cuantificado, pero no se puede eliminar.
- **Alcance.** El sistema se ha evaluado con una única asignatura, y como alternativa de pago se ha probado un único modelo.

7.4. Líneas de trabajo futuro

A partir de estas conclusiones y limitaciones se proponen las siguientes líneas de trabajo, ordenadas por el impacto esperado:

- **Mejorar la recuperación,** que es el principal límite del sistema. Por ejemplo, utilizar el modelo de reordenado bge-reranker-v2-m3, o probar técnicas para las preguntas de inferencia, como dividir las preguntas compuestas en varias búsquedas.
- **Validar la evaluación con personas,** pidiendo a personal cualificado de la asignatura que valoren una muestra de respuestas para comprobar hasta qué punto coinciden con el juez.
- **Evaluar conversaciones completas,** con un banco de diálogos que incluya preguntas de seguimiento.
- **Probar el sistema con estudiantes reales,** para medir su utilidad en el estudio y recoger sus opiniones.

- **Extender el sistema a otras asignaturas.** El sistema no depende del temario, bastaría con sustituir los documentos y el banco de preguntas.
- **Reducir el ruido de la medida,** ampliando el banco de preguntas, especialmente las de inferencia, y repitiendo cada evaluación varias veces.

Bibliografía

- [1] Similitud semántica, 2019. URL <https://livebook.manning.com/book/exploring-deep-learning-for-language/chapter-4#1>.
- [2] Embedding distancia, 2025. URL <https://medium.com/@gabrielpierobon/word-embeddings-la-piedra-angular-del-nlp-8dd2578f2381>.
- [3] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1:1–124, 2023. doi: 10.48550/arXiv.2303.18223.
- [4] Build an unstructured data pipeline for RAG | Databricks on AWS, 9 2026. URL <https://docs.databricks.com/aws/en/agents/retrieval-augmented-generation>.
- [5] Build an unstructured data pipeline for RAG | Databricks on AWS, 9 2026. URL <https://docs.databricks.com/aws/en/agents/tutorials/ai-cookbook/quality-data-pipeline-rag>.
- [6] Embedding models | LangChain, 9 2026. URL https://aidoczh.com/langchain/docs/concepts/embedding_models/embedding_models.html.
- [7] Rag basic pipeline, n.d. URL <https://medium.com/@drjulija/what-is-retrieval-augmented-generation-rag-938e4f6e03d1>.
- [8] Indexing, n.d. URL <https://docs.langchain.com/oss/javascript/langchain/rag>.
- [9] Retrieval and generation, n.d. URL <https://docs.langchain.com/oss/javascript/langchain/rag>.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Contexto, 2021. URL <https://arxiv.org/abs/2005.11401>.

- [11] Jakub Swacha and Michał Gracel. Retrieval-augmented generation (rag) chatbots for education: A survey of applications. *Applied Sciences*, 15(8), 2025. ISSN 2076-3417. doi: 10.3390/app15084234. URL <https://www.mdpi.com/2076-3417/15/8/4234>.
- [12] Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Project Management Institute, 7 edition, 2021. ISBN 978-1-62825-664-2.
- [13] Ministerio de Educación, Cultura y Deporte. Real Decreto 1125/2003, de 5 de septiembre, por el que se establece el sistema europeo de créditos y el sistema de calificaciones en las titulaciones universitarias de carácter oficial y validez en todo el territorio nacional. Boletín Oficial del Estado, núm. 224, de 18 de septiembre de 2003, 2003. URL <https://www.boe.es/eli/es/rd/2003/09/05/1125/con>.
- [14] Universidad de Jaén. Guía docente 13316001 - Trabajo fin de Grado. Grado en Ingeniería Informática (E.P.S. Jaén), curso 2025-26, 2025. URL <https://crea.ujaen.es/items/a5612264-36b9-4e70-b8ea-6fb54fd6c197/full>.
- [15] Ken Schwaber and Jeff Sutherland. *The Scrum Guide. The Definitive Guide to Scrum: The Rules of the Game*, 2020. URL <https://scrumguides.org/>.
- [16] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018. Systems and software engineering – Life cycle processes – Requirements engineering, 2018.
- [17] Escuela Politécnica Superior de Jaén. Normativa sobre Trabajos Fin de Grado, Fin de Máster y otros Trabajos Fin de Título de la EPSJ. Aprobada por la Junta de Escuela el 21 de julio de 2025, 2025. URL https://eps.ujaen.es/sites/centro_eps/files/uploads/documents/normativa/Normativa%20TFE%20EPSJ_Aprobada%2021Julio2025.pdf.
- [18] David M. Berry. The limits of computation: Joseph weizenbaum and the eliza chatbot. *Weizenbaum Journal of the Digital Society*, 3:1–24, 2023. doi: 10.34669/WI.WJDS/3.3.2.
- [19] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015. <http://neuralnetworksanddeeplearning.com/>.
- [20] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

- [21] Alex Sherstinsky. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D*, 404:1–2, 2019. doi: 10.1016/j.physd.2019.132306.
- [22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. doi: 10.48550/arXiv.1706.03762.
- [23] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Healthcare Informatics Research, 2016. ISBN 978-0262035613. doi: 10.4258/hir.2016.22.4.351.
- [24] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.
- [25] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020. doi: 10.48550/arXiv.2005.14165.
- [26] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8), 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- [27] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, et al. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023. doi: 10.48550/arXiv.2204.02311.
- [28] Ziyao Zhang and Chong Wang. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *ACM Digital Library*, 2:481–503, 2025. doi: 10.1145/3728894.
- [29] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020. doi: 10.48550/arXiv.2005.11401.

- [30] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024. doi: 10.48550/arXiv.2312.10997.
- [31] ChunkViz, 9 2026. URL <https://chunkviz.up.railway.app/>.
- [32] RecursiveCharacterTextSplitter | langchain_text_splitters, 9 2026. URL <https://reference.langchain.com/python/langchain-text-splitters/character/RecursiveCharacterTextSplitter>.
- [33] MTEB Leaderboard - a Hugging Face Space by mteb, 9 2026. URL <https://huggingface.co/spaces/mteb/leaderboard>.
- [34] Databricks Staff. What is a Vector Database?, 10 2024. URL <https://www.databricks.com/blog/what-is-vector-database>.
- [35] RAG chain for inference | Databricks on AWS, 9 2026. URL <https://docs.databricks.com/aws/en/agents/tutorials/ai-cookbook/fundamentals-inference-chain-rag>.
- [36] Improve RAG chain quality | Databricks on AWS, 9 2026. URL <https://docs.databricks.com/aws/en/agents/tutorials/ai-cookbook/quality-rag-chain>.
- [37] Assess performance: Metrics that matter | Databricks on AWS, 9 2026. URL <https://docs.databricks.com/aws/en/agents/tutorials/ai-cookbook/evaluate-assess-performance>.
- [38] Python Release Python 3.12.10, 2025. URL <https://www.python.org/downloads/release/python-31210/>.
- [39] Welcome to llamaindex, n.d. URL <https://developers.llamaindex.ai/python/framework/>.
- [40] deepset. Haystack: open-source AI orchestration framework, 2026. URL <https://haystack.deepset.ai/>.
- [41] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2310.03714.

- [42] Home - docs by langchain, n.d. URL <https://docs.langchain.com/>.
- [43] Google. Gemini models | Gemini API | Google AI for Developers, 2026. URL <https://ai.google.dev/gemini-api/docs/models>.
- [44] Ollama, 2026. URL <https://ollama.com/>.
- [45] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.
- [46] Helmholtz AI, Jülich Supercomputing Centre. Blablador API key access, 2026. URL https://sdlaml.pages.jsc.fz-juelich.de/ai/guides/blablador_api_access/.
- [47] OpenAI. GPT-5.1 Instant and GPT-5.1 Thinking System Card Addendum, 2025. URL <https://openai.com/index/gpt-5-system-card-addendum-gpt-5-1/>.
- [48] What is mupdf? - mupdf 1.28.2, n.d. URL <https://mupdf.readthedocs.io/en/1.28.2/guide/what-is-mupdf.html>.
- [49] Carlo Siebenschuh, Kyle Hippe, Ozan Gokdemir, Alexander Brace, Arham M Khan, Khalid Hossain, Yadu Babuji, Nicholas Chia, Venkatram Vishwanath, Arvind Ramanathan, et al. Adaparse: An adaptive parallel pdf parsing and resource scaling engine. *Proceedings of Machine Learning and Systems*, 7, 2025.
- [50] Unstructured Technologies. Unstructured: open-source libraries for pre-processing unstructured documents, 2026. URL <https://github.com/Unstructured-IO/unstructured>.
- [51] Datalab. Marker: Convert PDF to markdown + JSON quickly with high accuracy, 2026. URL <https://github.com/datalab-to/marker>.
- [52] Bin Wang, Chao Xu, Xiaomeng Zhao, Linke Ouyang, Fan Wu, Zhiyuan Zhao, Rui Xu, Kaiwen Liu, Yuan Qu, Fukai Shang, et al. MinerU: An open-source solution for precise document content extraction. *arXiv preprint arXiv:2409.18839*, 2024. doi: 10.48550/arXiv.2409.18839.
- [53] Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Fabian Lindlbauer, Kasper Dinkla, et al. Docling technical report. *arXiv preprint arXiv:2408.09869*, 2024. doi: 10.48550/arXiv.2408.09869.

- [54] Narayan S Adhikari and Shradha Agarwal. A comparative study of pdf parsing tools across diverse document categories. *arXiv preprint arXiv:2410.09871*, 2024.
- [55] Lukas Blecher, Guillem Cucurull, Thomas Scialom, and Robert Stojnic. Nougat: Neural optical understanding for academic documents. *arXiv preprint arXiv:2308.13418*, 2023. doi: 10.48550/arXiv.2308.13418.
- [56] Jake Poznanski, Jon Borchardt, Jason Dunkelberger, Regan Huff, Daniel Lin, Aman Rangapur, Christopher Wilhelm, Kyle Lo, and Luca Soldaini. olmOCR: Unlocking trillions of tokens in PDFs with vision language models. *arXiv preprint arXiv:2502.18443*, 2025. doi: 10.48550/arXiv.2502.18443.
- [57] Ahmed Nassar, Andres Marafioti, Matteo Omenetti, Maksym Lysak, Nikolaos Livathinos, Christoph Auer, Lucas Morin, et al. SmolDocling: An ultra-compact vision-language model for end-to-end multi-modal document conversion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. doi: 10.48550/arXiv.2503.11576.
- [58] Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar. DocLayNet: A large human-annotated dataset for document-layout segmentation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3743–3751, 2022. doi: 10.1145/3534678.3539043.
- [59] Ahmed Nassar, Nikolaos Livathinos, Maksym Lysak, and Peter Staar. TableFormer: Table structure understanding with transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4614–4623, 2022. doi: 10.48550/arXiv.2203.01017.
- [60] RapidAI. RapidOCR, 2026. URL <https://github.com/RapidAI/RapidOCR>.
- [61] Michael Günther, Isabelle Mohr, Daniel James Williams, Bo Wang, and Han Xiao. Late chunking: Contextual chunk embeddings using long-context embedding models. *arXiv preprint arXiv:2409.04701*, 2024. doi: 10.48550/arXiv.2409.04701.
- [62] Anthropic. Introducing Contextual Retrieval, 2024. URL <https://www.anthropic.com/news/contextual-retrieval>.
- [63] Renyi Qu, Ruixuan Tu, and Forrest Sheng Bao. Is semantic chunking worth the computational cost? In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025. doi: 10.48550/arXiv.2410.13070. URL <https://aclanthology.org/2025.findings-naacl.114/>.

- [64] Kenneth Enevoldsen, Isaac Chung, Imene Kerboua, Márton Kardos, Ashwin Mathur, David Stap, Jay Gala, Wissam Siblini, Dominik Krzemiński, Genta Indra Winata, et al. MMTEB: Massive multilingual text embedding benchmark. In *International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2502.13595.
- [65] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037, 2023. doi: 10.18653/v1/2023.eacl-main.148.
- [66] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019. doi: 10.18653/v1/D19-1410.
- [67] Sentence Transformers documentation, 2026. URL <https://sbert.net/>.
- [68] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual E5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*, 2024. doi: 10.48550/arXiv.2402.05672.
- [69] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024. doi: 10.48550/arXiv.2402.03216.
- [70] OpenAI. New embedding models and API updates, 2024. URL <https://openai.com/index/new-embedding-models-and-api-updates/>.
- [71] Jinhyuk Lee, Feiyang Chen, Sahil Dua, Daniel Cer, Madhuri Shanbhogue, Iftekhar Naim, et al. Gemini embedding: Generalizable embeddings from Gemini. *arXiv preprint arXiv:2503.07891*, 2025. doi: 10.48550/arXiv.2503.07891.
- [72] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025. doi: 10.48550/arXiv.2506.05176.
- [73] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024. doi: 10.48550/arXiv.2401.08281.

- [74] Qdrant. Qdrant documentation, 2026. URL <https://qdrant.tech/documentation/>.
- [75] Weaviate. Weaviate: the AI-native vector database, 2026. URL <https://weaviate.io/>.
- [76] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, pages 2614–2627, 2021. doi: 10.1145/3448016.3457550.
- [77] Pinecone Systems. Pinecone: the vector database, 2026. URL <https://www.pinecone.io/>.
- [78] pgvector: Open-source vector similarity search for Postgres, 2026. URL <https://github.com/pgvector/pgvector>.
- [79] Chroma. Chroma Docs, 2026. URL <https://docs.trychroma.com/>.
- [80] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.
- [81] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting in retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5303–5315, 2023. doi: 10.18653/v1/2023.emnlp-main.322.
- [82] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1762–1777, 2023. doi: 10.18653/v1/2023.acl-long.99.
- [83] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020. doi: 10.18653/v1/2020.emnlp-main.550.
- [84] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019.

- [85] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021. doi: 10.1145/3404835.3463098.
- [86] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020. doi: 10.1145/3397271.3401075.
- [87] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114.
- [88] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019. doi: 10.48550/arXiv.1901.04085.
- [89] cross-encoder/mmarco-mMiniLMv2-L12-H384-v1 | Hugging Face, 2026. URL <https://huggingface.co/cross-encoder/mmarco-mMiniLMv2-L12-H384-v1>.
- [90] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, pages 5776–5788, 2020. doi: 10.48550/arXiv.2002.10957.
- [91] Luiz Bonifacio, Vitor Jeronymo, Hugo Queiroz Abonizio, Israel Campiotti, Marzieh Fadaee, Roberto Lotufo, and Rodrigo Nogueira. mMARCO: A multilingual version of the MS MARCO passage ranking dataset. *arXiv preprint arXiv:2108.13897*, 2021. doi: 10.48550/arXiv.2108.13897.
- [92] BAAI. BAAI/bge-reranker-v2-m3 | Hugging Face, 2026. URL <https://huggingface.co/BAAI/bge-reranker-v2-m3>.
- [93] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 14918–14937, 2023. doi: 10.18653/v1/2023.emnlp-main.923.
- [94] Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st*

- Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998. doi: 10.1145/290941.291025.
- [95] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12: 157–173, 2024. doi: 10.1162/tacl_a_00638.
- [96] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. doi: 10.48550/arXiv.2407.21783.
- [97] Gemini Team, Google. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023. doi: 10.48550/arXiv.2312.11805.
- [98] OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025. doi: 10.48550/arXiv.2508.10925.
- [99] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. doi: 10.48550/arXiv.2505.09388.
- [100] Swiss AI Initiative. Apertus: Democratizing open and compliant LLMs for global language environments. *arXiv preprint arXiv:2509.14233*, 2025. doi: 10.48550/arXiv.2509.14233.
- [101] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.48550/arXiv.2404.13076.
- [102] Google. NotebookLM, 2026. URL <https://notebooklm.google/>.
- [103] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 311–318, 2002. doi: 10.3115/1073083.1073135.
- [104] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, 2004. URL <https://aclanthology.org/W04-1013/>.
- [105] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020. doi: 10.48550/arXiv.1904.09675.

- [106] TruLens: evaluation and tracking for LLM experiments, 2026. URL <https://www.trulens.org/>.
- [107] Confident AI. DeepEval: the LLM evaluation framework, 2026. URL <https://github.com/confident-ai/deepeval>.
- [108] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023. doi: 10.48550/arXiv.2309.15217.
- [109] Ragas documentation, 2026. URL <https://docs.ragas.io/>.
- [110] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.48550/arXiv.2306.05685.
- [111] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.
- [112] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [113] Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937. doi: 10.1080/01621459.1937.10503522.
- [114] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.

